

Hydra: Local Enterprise LLM Farm Architecture

Platform Engineering

2026-03

Hydra: Local Enterprise LLM Farm — Technical Architecture Specification

Table of Contents

1. Hardware Baseline and Hard Constraints
 - 1.1 Cluster Inventory
 - 1.2 Chip-Generation Performance Characteristics
 - 1.3 Per-Node Memory Budget
 - 1.4 Absolute Hardware Constraints
2. System Architecture
 - 2.1 High-Level Component Topology
 - 2.2 Mermaid Architecture Diagram
 - 2.3 Component Responsibilities
 - 2.4 Request Lifecycle
 - 2.5 Node Assignment by Chip Generation
3. KV Cache Math and Concurrent Request Capacity
 - 3.1 KV Cache Formula
 - 3.2 Per-Model KV Cache Rates
 - 3.3 Per-Node Memory Partitioning
 - 3.4 Concurrent Request Capacity Per Node
 - 3.5 Cluster-Level Capacity Estimate
 - 3.6 Context Window Enforcement Derived from KV Math
4. Model Family Analysis and Maximum Feasible Sizes
 - 4.1 Llama Family
 - 4.2 Qwen Family
 - 4.3 Mistral / Mistral AI Family
 - 4.4 Mixtral (Mixture of Experts) Family
 - 4.5 BitNet (Research Models)
 - 4.6 Maximum Feasible Model Sizes — Summary
 - 4.7 Model Selection Recommendations Per Pool

- 5. Inference Runtime Comparison and Recommendation
 - 5.1 Runtime Feature Matrix
 - 5.2 Runtime Detailed Analysis
 - 5.3 Recommended Runtime Stack
- 6. Distributed Inference Analysis
 - 6.1 Distributed Inference on Apple Silicon: Mechanism
 - 6.2 Network Constraints
 - 6.3 KV Cache Behavior in Distributed Mode
 - 6.4 EXO Benchmark Data (Mac Mini Clusters)
 - 6.5 Feasibility Matrix for Target Model Sizes
 - 6.6 Distributed Inference Operational Rules
 - 6.7 Why Tensor Parallelism Is Not Viable (Detailed)
- 7. Quantization Strategy and Policy
 - 7.1 GGUF Quantization Mechanics
 - 7.2 Quantization Reference Table — 7B Parameter Model
 - 7.3 Quantization Reference Table — By Model Size Class
 - 7.4 Quantization Governance Policy
 - 7.5 MLX Quantization Format Note
- 8. Node Pool Allocation and Model Tiering
 - 8.1 Pool Design — 40 Inference Node Cluster
 - 8.2 Chip-to-Pool Assignment Matrix
 - 8.3 Model Tiering Strategy
 - 8.4 Task Category to Pool Routing Map
 - 8.5 Model Loading and Node Startup
- 9. Governance and Policy Enforcement
 - 9.1 Governance Layers
 - 9.2 User Group Definitions
 - 9.3 Governance Policy Schema (Full Reference)
 - 9.4 Enforcement Execution Path
 - 9.5 Rate Limit Counter Design
 - 9.6 Context Window Governance Architecture
- 10. Multimodal Capability and Isolation
 - 10.1 Supported Modality Pools
 - 10.2 Vision Model Isolation Design
 - 10.3 Speech Model Isolation Design
 - 10.4 Embedding Pool Design
- 11. Storage Layer
 - 11.1 Storage Architecture Overview
 - 11.2 MinIO Model Store
 - 11.3 Model Registry Database
 - 11.4 Node-Local Model Cache
 - 11.5 Model Approval Workflow

12. Networking Model
12.1 Network Topology
12.2 Required Network Bandwidth
12.3 Service Discovery
12.4 TLS and Certificate Management
12.5 Port Reference
13. Failure Handling and Recovery
13.1 Failure Mode Matrix
13.2 Request-Level Failure Handling
13.3 Graceful Shutdown Procedure
13.4 Pool-Level Capacity Degradation Thresholds
14. Security Controls
14.1 Authentication and Authorization
14.2 Network Security
14.3 Model Integrity
14.4 Audit Logging
14.5 Secrets Management
14.6 Software Supply Chain
15. Monitoring and Observability
15.1 Observability Stack Architecture
15.2 Complete Metrics Reference
15.3 Grafana Dashboard Specifications
15.4 Alert Rules
15.5 Log Management
16. Throughput Expectations
16.1 Per-Node Throughput Baselines
16.2 Cluster-Level Throughput Summary
16.3 User Experience Latency Targets
16.4 Capacity Expansion Path
Appendix A: Technology Stack Summary
Appendix B: Operational Runbook Reference
B.1 Adding a New Inference Node
B.2 Rolling Model Update
Appendix C: Capacity Reference Card

Hydra: Local Enterprise LLM Farm — Technical Architecture Specification

Platform: 40-node Apple Silicon Mac Mini cluster

Scale: $\leq 1,000$ internal users

Deployment class: On-premises, air-gapped-capable

Revision: 1.0 — 2026-03

Table of Contents

1. [Hardware Baseline and Hard Constraints](#)
 2. [System Architecture](#)
 3. [KV Cache Math and Concurrent Request Capacity](#)
 4. [Model Family Analysis and Maximum Feasible Sizes](#)
 5. [Inference Runtime Comparison and Recommendation](#)
 6. [Distributed Inference Analysis](#)
 7. [Quantization Strategy and Policy](#)
 8. [Node Pool Allocation and Model Tiering](#)
 9. [Governance and Policy Enforcement](#)
 10. [Multimodal Capability and Isolation](#)
 11. [Storage Layer](#)
 12. [Networking Model](#)
 13. [Failure Handling and Recovery](#)
 14. [Security Controls](#)
 15. [Monitoring and Observability](#)
 16. [Throughput Expectations](#)
-

1. Hardware Baseline and Hard Constraints

1.1 Cluster Inventory

Attribute	Value
Node count	40 × Mac Mini
Chip variants present	M1, M3, M4 (base variants only)
Unified RAM per node	16 GB
CPU cores per node	10 (typical)
GPU cores per node	8–10 (Metal, Apple GPU)
Neural Engine	16-core NPU per node
Local storage	256–512 GB NVMe SSD per node
Network	1 Gbps Ethernet (LAN)
Max users	1,000 internal
Environment	On-premises, no cloud dependency

1.2 Chip-Generation Performance Characteristics

Apple Silicon memory bandwidth is the primary constraint on decode throughput. All three chip generations are present in the cluster. The scheduler **MUST** account for this heterogeneity when assigning workloads to node pools.

Chip	Memory Bandwidth	GPU Cores	~tok/s on 8B Q4_K_M	Thermal class
M1	68.25 GB/s	7–8	15–20	Lower
M3	102.4 GB/s	8–10	22–30	Moderate
M4	120.0 GB/s	8–10	25–35	Moderate

M1 vs M4 decode gap: ~75%. Route latency-sensitive workloads to M4 nodes exclusively. M1 nodes absorb embedding, speech, and overflow batching workloads where bandwidth is not the bottleneck.

1.3 Per-Node Memory Budget

Unified memory is shared between CPU, GPU, and Neural Engine. macOS retains a baseline allocation for kernel, graphics compositor, and system daemons that cannot be reclaimed.

Allocation	Size
Total unified RAM	16,384 MB
macOS + system daemons (baseline)	~2,500 MB
vLLM-MLX runtime overhead	~500 MB
Available for model weights + KV cache	~13,400 MB
Model weight ceiling (60% rule)	9,600 MB
KV cache budget (remaining after weights)	~3,800–8,600 MB (varies by model)

60% rule rationale: Long-context inference causes KV cache to grow linearly with sequence length. Reserving 40% of available headroom prevents the OS page daemon from evicting model weights mid-request, which causes 10–100x latency spikes. This is a hard operational limit enforced by the runtime configuration, not a soft guideline.

1.4 Absolute Hardware Constraints

The following constraints are architectural facts of the platform. They cannot be worked around and must be treated as design axioms:

- **No NCCL:** NVIDIA Collective Communications Library does not exist on this hardware. Tensor-parallel collective operations (AllReduce, AllGather) have no low-latency equivalent across Mac nodes.
- **No GPU-GPU RDMA:** Apple Silicon has no NVLink equivalent. Cross-node GPU communication goes through Ethernet.
- **No CUDA:** All GPU compute runs through Metal or MLX. vLLM's CUDA-dependent PagedAttention kernel is not available. Only vLLM-MLX and vLLM-Metal (MLX backend) are viable.
- **Unified memory ceiling:** 16 GB is the absolute bound. No ECC, no memory pooling across nodes at hardware level.
- **Metal architecture:** Apple's Metal API exposes GPU compute but does not support multi-device unified memory across nodes.
- **Thermal throttling:** Sustained inference workloads on all 10 GPU cores can trigger thermal throttling on Mac Mini, reducing effective bandwidth by 5–15%. Node pools must account for sustained vs. burst workload profiles.

2. System Architecture

2.1 High-Level Component Topology



2.2 Mermaid Architecture Diagram

flowchart TD

```

Clients["Clients (up to 1000 users)"] -->|"HTTPS"| HAP["HAProxy + keepalived"]
HAP --> GW1["LiteLLM Gateway 1"]
HAP --> GW2["LiteLLM Gateway 2"]

GW1 & GW2 -->|"tag=fast"| FastPool["Fast Pool\n20 nodes / 8B Q4"]
GW1 & GW2 -->|"tag=reasoning"| ReasonPool["Reasoning Pool\n8 nodes / 14B Q4"]
GW1 & GW2 -->|"tag=vision"| VisionPool["Vision Pool\n6 nodes / 8B VL Q4"]
GW1 & GW2 -->|"tag=embed"| EmbedPool["Embedding Pool\n4 nodes / BGE-M3"]
GW1 & GW2 -->|"tag=speech"| SpeechPool["Speech Pool\n2 nodes / Whisper"]

FastPool & ReasonPool & VisionPool & EmbedPool & SpeechPool --> ModelStore["Model Store\nMinIO / NFS"]
FastPool & ReasonPool & VisionPool & EmbedPool & SpeechPool --> Prom["Prometheus"]
Prom --> Grafana["Grafana + Alertmanager"]

```

2.3 Component Responsibilities

HAProxy + keepalived

- Active/passive pair using keepalived VIP (Virtual IP).
- TCP load-balancing to LiteLLM gateway instances on port 4000.
- Health check: HTTP GET `/health` every 5s, 3 failures = node removed.
- Deployed on 2 dedicated Mac Mini nodes (not part of inference pool, M1 acceptable).

LiteLLM Gateway

- **Role:** OpenAI-compatible API endpoint, policy enforcement, routing, budget tracking.
- **Deployed:** 2 active instances + 1 cold spare. Each instance is stateless except for in-memory rate-limit counters (shared via Redis sidecar).
- **Responsibilities:**
 - Authenticate requests (OIDC/API key → group membership).
 - Enforce per-user and per-group token budgets and rate limits.
 - Classify request task category (via `tags` field in request or metadata header).
 - Select target pool based on task category and user group ACL.
 - Within pool: route using least-busy strategy (fewest active requests).
 - Count input tokens using model-specific tokenizer before forwarding.
 - Reject requests exceeding context window policy.
 - Log structured request records to persistent audit store.

vLLM-MLX Runtime (per inference node)

- **Role:** Inference worker. One instance per node.

- **Responsibilities:**

- Serve one model per node (no multi-model colocation on inference nodes).
- Continuous batching via vLLM-MLX scheduler.
- Paged KV cache management.
- Expose OpenAI-compatible HTTP API on port 8000 (internal only).
- Export Prometheus metrics on port 9090.
- Enforce max KV cache size via `--max-model-len` and `--gpu-memory-utilization` flags.

Model Store

- **Role:** Authoritative source for approved model artifacts.
- **Implementation:** MinIO (S3-compatible) on a dedicated node pair (primary + replica). NFS is acceptable for simpler setups.
- **Contents:** GGUF and MLX-format weight files, metadata JSON (version, sha256, approved_by, pool_assignment).

Model Registry

- **Role:** Metadata catalog of approved models. Separate from the artifact store.
- **Implementation:** PostgreSQL (single instance, replicated) or SQLite for small deployments.

Redis (sidecar to gateways)

- **Role:** Shared rate-limit counters and token budget state between LiteLLM gateway instances.
- **Implementation:** Single Redis instance (or Redis Sentinel pair for HA).

Observability Stack

- **Prometheus:** Scrapes all inference nodes, gateways, HAProxy, and node exporters.
- **Grafana:** Dashboards for cluster overview, per-pool metrics, user/group usage, capacity planning.
- **Alertmanager:** Routes alerts (Slack, email, PagerDuty) for node failures, memory pressure, latency breaches, and budget exhaustion.

2.4 Request Lifecycle

1. Client sends POST /v1/chat/completions (OpenAI format)
2. HAProxy routes to LiteLLM GW (active instance)
3. LiteLLM GW:
 - a. Authenticates API key → resolves user + group
 - b. Checks token budget (Redis counter) → reject if exhausted
 - c. Checks rate limit (Redis counter) → reject if exceeded
 - d. Reads task tag from request metadata or applies classifier
 - e. Validates user group has ACL for requested pool/model
 - f. Counts input tokens → reject if > context_limit - max_gen_tokens
 - g. Selects target pool → selects least-busy node in pool
 - h. Forwards request to inference node (HTTP)
4. Inference node (vLLM-MLX):
 - a. Adds request to continuous batch scheduler
 - b. Allocates KV cache pages
 - c. Runs prefill + decode
 - d. Streams tokens back (SSE) or returns full response
5. LiteLLM GW:
 - a. Records output tokens consumed → increments budget counter
 - b. Writes audit log record
 - c. Returns response to client

2.5 Node Assignment by Chip Generation

Given heterogeneous chip mix, node pool assignment is chip-aware:

Pool	Preferred chip	Rationale
Fast text	M4 (primary), M3 (fill)	Decode bandwidth critical
Reasoning	M4 (primary), M3 (fill)	Long decode sequences, high bandwidth needed
Vision	M3 or M4	Image preprocessing + token decode mixed
Embedding	M1 acceptable	Encoder-only, bandwidth-insensitive
Speech	M1 acceptable	NPU/CPU-bound, bandwidth not the bottleneck
HAProxy/ Gateway/ Monitoring	M1	Not inference; minimal compute

In a typical 40-node deployment with mixed chip generations, allocate M4 nodes to Fast and Reasoning pools first. If the cluster is predominantly one generation (e.g., all M4), the allocation above still holds for workload prioritization purposes.

3. KV Cache Math and Concurrent Request Capacity

KV cache sizing is the primary variable that determines per-node concurrency. Model weight size determines whether a model fits; KV cache determines how many simultaneous users it can serve. These two together consume the full available memory budget.

3.1 KV Cache Formula

$$\text{KV_bytes_per_token} = 2 \times \text{num_layers} \times \text{num_kv_heads} \times \text{head_dim} \times \text{bytes_per_element}$$

- Factor of 2: separate Key tensor and Value tensor per layer.
- `bytes_per_element`: FP16 = 2, FP32 = 4. vLLM-MLX stores KV cache in FP16 by default.
- KV cache grows **linearly** with sequence length and **linearly** with batch size.

3.2 Per-Model KV Cache Rates

Model	Layers	KV heads	Head dim	dtype	Bytes/token	MB/token
Llama 3.1 8B	32	8 (GQA)	128	FP16	131,072	0.125
Qwen 2.5 7B	28	4 (GQA)	128	FP16	57,344	0.055
Qwen 2.5 14B	48	8 (GQA)	128	FP16	196,608	0.188
Mistral 7B	32	8 (GQA)	128	FP16	131,072	0.125
Mistral Nemo 12B	40	8 (GQA)	128	FP16	163,840	0.156
MiniCPM-V 2.6 (8B)	32	8 (GQA)	128	FP16	131,072	0.125

Example derivation — Llama 3.1 8B:

$$2 \times 32 \text{ layers} \times 8 \text{ KV-heads} \times 128 \text{ head-dim} \times 2 \text{ bytes} = 131,072 \text{ bytes} = 0.125 \text{ MB/token}$$

At 4K context, one request occupies: $4096 \times 0.125 \text{ MB} = 512 \text{ MB}$ of KV cache.

3.3 Per-Node Memory Partitioning

Node configuration: Llama 3.1 8B Q4_K_M (primary fast pool model)

```

Total RAM           : 16,384 MB
macOS + daemons    :  2,500 MB
Runtime overhead   :   500 MB
-----
Available          : 13,384 MB
Model weights (Q4_K_M) :  4,800 MB
-----
KV cache budget    :  8,584 MB

```

Node configuration: Qwen 2.5 14B Q4_K_M (reasoning pool model)

```

Total RAM           : 16,384 MB
macOS + daemons    :  2,500 MB
Runtime overhead   :   500 MB
-----
Available          : 13,384 MB
Model weights (Q4_K_M) :  9,000 MB
-----
KV cache budget    :  4,384 MB

```

3.4 Concurrent Request Capacity Per Node

Llama 3.1 8B Q4_K_M — KV budget: 8,584 MB — 0.125 MB/token

Context limit	KV per request	Theoretical max	Operational target
1K tokens	125 MB	~68	40
2K tokens	256 MB	~33	20
4K tokens	512 MB	~16	10
8K tokens	1,024 MB	~8	5
16K tokens	2,048 MB	~4	2

Operational target = theoretical × 0.60 (headroom for prefill spikes and OS jitter).

Qwen 2.5 14B Q4_K_M — KV budget: 4,384 MB — 0.188 MB/token

Context limit	KV per request	Theoretical max	Operational target
1K tokens	188 MB	~23	14
2K tokens	375 MB	~11	7
4K tokens	750 MB	~5	3

Qwen 2.5 7B Q4_K_M — KV budget: 8,884 MB — 0.055 MB/token

Note: Qwen 2.5 7B uses GQA with only 4 KV heads, making its KV cache significantly smaller than Llama 3.1 8B despite similar parameter count.

Context limit	KV per request	Theoretical max	Operational target
4K tokens	220 MB	~40	24
8K tokens	440 MB	~20	12
16K tokens	880 MB	~10	6

Key insight: Qwen 2.5 7B’s architecture is significantly more KV-cache-efficient than Llama 3.1 8B at equivalent context lengths. For high-concurrency chat workloads, Qwen 2.5 7B is the superior model choice per node.

3.5 Cluster-Level Capacity Estimate

Fast pool: 20 nodes × Llama 3.1 8B Q4_K_M, 4K default context

```
20 nodes × 10 operational concurrent = 200 simultaneous requests
Peak demand estimate: 1,000 users × 10% concurrency = 100 concurrent
Headroom factor: 200 / 100 = 2.0×
```

Fast pool: 20 nodes × Qwen 2.5 7B Q4_K_M, 4K default context

```
20 nodes × 24 operational concurrent = 480 simultaneous requests
Peak demand: 100 concurrent
Headroom factor: 4.8×
```

Reasoning pool: 8 nodes × Qwen 2.5 14B Q4_K_M, 2K default context

```
8 nodes × 7 operational concurrent = 56 simultaneous requests
Expected reasoning demand: 1,000 users × 5% concurrency × 30% use reasoning = 15 concurrent
Headroom factor: 3.7×
```

Design conclusion: The cluster is comfortably over-provisioned for the stated 1,000-user load at the default context sizes. Capacity can absorb context expansion (8K+ for premium users) without exhaustion.

3.6 Context Window Enforcement Derived from KV Math

Policy max_context values below are derived from the KV budget calculations above, not arbitrary.

Pool	Model	Max context enforced	Rationale
Fast	Llama 3.1 8B Q4	8,192 tokens	8 concurrent at 8K within budget
Fast (high-concurrency)	Qwen 2.5 7B Q4	16,384 tokens	6 concurrent at 16K within budget
Reasoning	Qwen 2.5 14B Q4	4,096 tokens	3 concurrent at 4K within budget
Vision	MiniCPM-V 2.6	2,048 text tokens + image	Image tokens inflate KV budget
Embedding	BGE-M3	8,192 tokens	Encoder-only, no KV cache growth

4. Model Family Analysis and Maximum Feasible Sizes

4.1 Llama Family

Vendor: Meta AI. **License:** Llama 3 Community License (permissive for most commercial use).

Model	Params	Architecture	FP16 size	Q4_K_M size	Q5_K_M size	Q8_0 size	Apple Silicon	Expected tokens (M4, Q4)
Llama 3.2 1B	1B	Transformer, GQA	~2 GB	~0.7 GB	~0.9 GB	~1.1 GB	Full (MLX, llama.cpp)	80–120
Llama 3.2 3B	3B	Transformer, GQA	~6 GB	~2.0 GB	~2.4 GB	~3.5 GB	Full	50–80
Llama 3.1 8B	8B	Transformer, GQA	~15 GB	~4.8 GB	~5.5 GB	~8.2 GB	Full	25–35
Llama 3.1 70B	70B	Transformer, GQA	~140 GB	~40 GB	~48 GB	~72 GB	Not single-node	2–5 (distributed)
Llama 3.1 405B	405B	Transformer, GQA	~810 GB	~230 GB	—	—	Not feasible	—

Quantization options: Full GGUF range (Q2_K through Q8_0) via llama.cpp; 4-bit and 8-bit via MLX quantize.

Apple Silicon compatibility: Full. MLX-native implementations available. Metal acceleration works for all sizes that fit in RAM.

Single-node verdict: Llama 3.1 8B at Q4_K_M is the primary fast-pool model. All sizes $\leq 8\text{B}$ are comfortable.

4.2 Qwen Family

Vendor: Alibaba Cloud / Qwen Team. License: Apache 2.0 (most sizes), Tongyi Qianwen License (72B).

Model	Params	FP16 size	Q4_K_M size	Q8_0 size	KV heads	Max context (native)	Apple Silicon
Qwen 2.5 0.5B	0.5B	~1 GB	~0.3 GB	~0.5 GB	2	32K	Full
Qwen 2.5 1.5B	1.5B	~3 GB	~1.0 GB	~1.6 GB	2	32K	Full
Qwen 2.5 3B	3B	~6 GB	~2.0 GB	~3.5 GB	2	32K	Full
Qwen 2.5 7B	7B	~14 GB	~4.5 GB	~7.5 GB	4	128K	Full
Qwen 2.5 14B	14B	~28 GB	~9.0 GB	~15 GB	8	128K	Q4 only (tight)
Qwen 2.5 32B	32B	~64 GB	~19 GB	~33 GB	8	128K	Not single-node
Qwen 2.5 72B	72B	~144 GB	~43 GB	~72 GB	8	128K	Not single-node
Qwen 2.5 Coder 7B	7B	~14 GB	~4.5 GB	~7.5 GB	4	128K	Full
Qwen-VL 7B	7B	~14 GB	~4.8 GB	~8 GB	4	32K + image	Full (vision)

Key property: Qwen 2.5's GQA configuration uses far fewer KV heads than Llama 3 equivalents. The 7B uses 4 KV heads (vs Llama's 8), halving KV cache memory. This enables significantly higher concurrency per node.

Single-node verdict: Qwen 2.5 7B and Qwen 2.5 Coder 7B are primary fast-pool candidates. Qwen 2.5 14B is feasible at Q4_K_M with context capped at $\leq 4\text{K}$.

4.3 Mistral / Mistral AI Family

Vendor: Mistral AI. License: Apache 2.0.

Model	Params	FP16 size	Q4_K_M size	Q5_K_M size	Q8_0 size	Apple Silicon
Mistral 7B v0.3	7B	~14 GB	~4.5 GB	~5.0 GB	~7.5 GB	Full
Mistral Nemo 12B	12B	~24 GB	~7.6 GB	~8.8 GB	~13.1 GB	Q4 fits (marginal KV)
Mistral Small 22B	22B	~44 GB	~13 GB	~15 GB	~22 GB	Not single-node
Mistral Large 2 (123B)	123B	~246 GB	~69 GB	—	—	Not feasible

Single-node verdict: Mistral 7B v0.3 fits comfortably. Mistral Nemo 12B at Q4_K_M uses 7.6 GB for weights, leaving ~5.7 GB for KV cache — marginal for concurrent use. Recommended only for reasoning pool with strict 2K context cap.

4.4 Mixtral (Mixture of Experts) Family

Vendor: Mistral AI. Architecture: Sparse MoE — 8 expert FFN blocks, 2 active per token.

Model	Total params	Active params	FP16 size	Q4_K_M size	Q2_K size	Apple Silicon
Mixtral 8×7B	46.7B	~12B active	~90 GB	~26 GB	~16 GB	Not single-node
Mixtral 8×22B	141B	~39B active	~280 GB	~80 GB	—	Not feasible

MoE constraint: Although only ~12B parameters are active during inference, all 46.7B parameters must reside in memory. The full Q4_K_M file size is ~26 GB, exceeding 16 GB RAM. Even Q2_K (~16 GB) exceeds the safe weight budget after OS overhead.

Single-node verdict: Mixtral 8×7B is **not feasible** on any 16 GB node at any standard quantization. It would require a minimum of 2 nodes with pipeline sharding. Even distributed, quality at Q2_K is degraded. **Not recommended for this cluster.**

4.5 BitNet (Research Models)

Vendor: Microsoft Research. Architecture: 1-bit / ternary weights ({-1, 0, +1}).

Model	Params	Weight precision	Effective size	Apple Silicon	tok/s (ARM CPU)	Status
BitNet b1.58 2B-4T	2B	1-bit ternary	~0.4 GB	Full (ARM kernels)	30–60	Research/experimental
BitNet b1.58 3B	3B	1-bit ternary	~0.6 GB	Full (ARM kernels)	20–40	Research/experimental
BitNet 100B (theoretical)	100B	1-bit	~12.5 GB	CPU-only	5–7	Experimental

BitNet mechanics: Weights are stored as ternary values (1.58 bits per weight). At runtime, matrix multiplications are replaced by additions, reducing FLOPs dramatically. Memory footprint is ~16× smaller than FP16 equivalents.

Apple Silicon notes: BitNet uses ARM-optimized CPU kernels. Metal GPU acceleration is not currently supported — inference runs on the CPU. The M4's efficiency cores and Neural Engine do not directly accelerate BitNet operations through official channels.

Single-node verdict: BitNet fits trivially. However: (1) ecosystem maturity is low — limited fine-tuned variants; (2) GPU acceleration not available; (3) quality at 2B–3B parameters is below Llama 3.1 8B Q4 at comparable memory. **Suitable for edge/embedded use cases or experimental evaluation, not primary serving.**

4.6 Maximum Feasible Model Sizes — Summary

Category	Size range	Notes
Single-node comfort zone	7B–8B at Q4_K_M	Weights 4.5–4.8 GB; ample KV budget; primary fleet model
Single-node tight fit	12B–14B at Q4_K_M	Weights 7.6–9.0 GB; strict ≤4K context; ≤5 concurrent
Single-node absolute limit	~14B at Q4_K_M	9 GB weights + 4.4 GB KV = 13.4 GB; no further headroom
Distributed only (experimental)	30B at Q4_K_M	3–4 nodes via pipeline sharding; 5–10 tok/s single-request
Not feasible on this cluster	≥32B single-node, Mixtral 8×7B, 70B+ practical	Exceeds RAM at any workable quantization

4.7 Model Selection Recommendations Per Pool

Pool	Primary model	Alternate model	Selection rationale
Fast text	Qwen 2.5 7B Q4_K_M	Llama 3.1 8B Q4_K_M	Qwen's 4 KV-heads enable higher concurrency
Coding	Qwen 2.5 Coder 7B Q4_K_M	DeepSeek Coder 6.7B Q4_K_M	Code-specialized training
Reasoning	Qwen 2.5 14B Q4_K_M	Qwen 2.5 7B Q4_K_M	Larger model for multi-step reasoning
Vision	MiniCPM-V 2.6 (8B) Q4_K_M	Qwen-VL 7B Q4_K_M	MiniCPM-V produces fewer image tokens (640 vs 2000+)
Embedding	BGE-M3 (569M) FP16	BGE-Large-EN FP16	BGE-M3: multilingual, 8K context, sparse+dense
Speech	Whisper Large v3 Turbo (1B compressed)	Whisper Medium	WhisperKit ANE acceleration, 0.46s real-time latency

5. Inference Runtime Comparison and Recommendation

5.1 Runtime Feature Matrix

Runtime	Metal / MLX accel	Continuous batching	Paged KV cache	Multi-node	Full GGUF range	Multimodal
vLLM-MLX	Yes — MLX native	Yes	Yes (paged attention)	No (single-node only)	No (MLX 4-bit only)	Yes (vision, audio, video)
vLLM-Metal	Yes — MLX backend	Yes	Yes (paged attention)	No	Limited	No — text only
Ollama	Yes — Metal via llama.cpp	Partial (OLLAMA_NUM_PARALLEL)	No	No	Yes — all GGUF types	Yes — LLaVA, Qwen-VL
llama.cpp	Yes — Metal	Server mode (--parallel)	No	No	Yes — all GGUF types	Partial (clip/llava)
MLX (raw mlx-lm)	Yes — native	Via mlx-lm serve	No	Community only (mlx_sharding)	Via mlx.core.quantize	Yes
LM Studio	Yes — Metal	No	No	No	Yes — GGUF	Limited
EXO	Yes — MLX backend	Pipeline parallel	No	Yes — pipeline sharding	Via MLX	Limited

5.2 Runtime Detailed Analysis

vLLM-MLX

- **Architecture:** Built on Apple's MLX framework. vLLM scheduling engine (continuous batching, preemption, recompute) wired to MLX compute backend. Zero-copy operations via unified memory.
- **Continuous batching:** Full implementation. New requests join the batch mid-decode. No head-of-line blocking. Critical for multi-tenant serving.
- **Paged KV cache:** Block-based KV allocation eliminates memory fragmentation. Prefix caching supported — 28× speedup on repeated image queries.

- **Throughput:** Benchmarked at 21–87% higher throughput than llama.cpp across 0.6B–30B models. 4.3× aggregate throughput improvement at 16 concurrent requests vs. single-request mode.
- **Peak speed:** 525 tokens/sec on M4 Max for text models; 400+ tokens/sec on M4 standard.
- **Multimodal:** Text, image, video, audio — broadest multimodal coverage of any Apple Silicon runtime.
- **Limitation:** 4-bit quantization only (MLX format, not GGUF). No GGUF Q3/Q5/Q8 support. Text-only variant (vLLM-Metal) also available with same batching/KV cache features.
- **Installation:** pip install + source build (Rust toolchain required). Shell script installer available.
- **API:** OpenAI-compatible. Supports streaming (SSE), function calling, structured output.

vLLM-Metal

- **Architecture:** Community plugin under official vLLM project. MLX inference + PyTorch model loading. Developed in collaboration with Docker.
- **vs. vLLM-MLX:** First-party organizational backing, but text-only. vLLM-MLX has more features and faster installation. Use vLLM-Metal if official project affiliation is a compliance requirement.
- **Limitation:** Text-only. Cannot serve vision, audio, or video models.

Ollama

- **Architecture:** llama.cpp backend with Metal acceleration. Go server managing model lifecycle. GGUF model format.
- **Concurrency:** `OLLAMA_NUM_PARALLEL` controls max parallel requests per model. Default varies by available memory. Does NOT create multiple model instances — one copy in memory, parallel decode.
- **Multi-model:** `OLLAMA_MAX_LOADED_MODELS` allows multiple models resident simultaneously, but model eviction occurs when memory is exhausted. For single-model-per-node design, this is not relevant.
- **Limitation:** No paged KV cache. Memory fragmentation under sustained load. Single-instance-per-model constraint means horizontal scaling requires node-level routing (which the LiteLLM gateway provides). Model loading blocks in-flight requests to other models.
- **Use case in this stack:** Fallback for vision models not yet supported by vLLM-MLX. Acceptable for low-concurrency speech-adjacent workloads.

llama.cpp

- **Architecture:** C++ inference engine. Metal backend for Apple GPU acceleration. GGUF native format.
- **Server mode:** `llama-server --parallel N` enables N-way parallel request handling. No true continuous batching — batch is fixed at request start. Under sustained load, head-of-line blocking occurs.

- **Quantization coverage:** Widest GGUF range — Q2_K through Q8_0, K-quants, IQ-quants, FP16.
- **Limitation:** No paged KV cache. No continuous batching. Performance degrades at high concurrency vs. vLLM-MLX.
- **Use case:** Suitable for single-user or very-low-concurrency scenarios. Not recommended for multi-tenant production nodes in this cluster.

MLX Raw (mlx-lm)

- **Architecture:** Apple's MLX framework with mlx-lm serving layer. Native Metal acceleration with zero-copy unified memory.
- **Batching:** `mlx-lm` includes a continuous batching server. Performance competitive with vLLM-MLX for single-model serving.
- **Distributed:** Community implementations (`mlx_sharding`, `mlx-distributed-ring-inference`) provide pipeline parallelism. Not production-grade. Use EXO for distributed workloads.
- **Limitation:** Less battle-tested for multi-tenant workloads than vLLM-MLX. Paged KV cache not available.

LM Studio

- **Architecture:** Desktop GUI application for local LLM management. Metal backend.
- **Limitation:** Not designed for headless server deployment. No REST API server mode in production configuration. Not suitable for this cluster.
- **Verdict:** Excluded from production stack consideration.

EXO

- **Architecture:** Peer-to-peer distributed inference framework. Pipeline parallelism with automatic peer discovery. MLX backend on Apple Silicon.
- **Multi-node:** Only framework with production-tested Mac Mini cluster support. Demonstrated: DeepSeek V3 671B on 8× M4 Pro 64 GB cluster at 5.37 tok/s; 3× M4 Pro cluster at 108.8 agg tok/s for multi-request 3B workloads.
- **Latency behavior:** Single-request latency increases with node count due to sequential pipeline stages. Multi-request throughput scales better.
- **Limitation:** Experimental. No paged KV cache. No governance hooks. Not suitable as primary serving stack.
- **Use case:** Experimental distributed pool for 30B models only.

5.3 Recommended Runtime Stack

Component	Runtime	Rationale
Fast text pool (20 nodes)	vLLM-MLX	Continuous batching + paged KV cache = mandatory for 10–24 concurrent/node
Reasoning pool (8 nodes)	vLLM-MLX	Same — continuous batching required
Vision pool (6 nodes)	vLLM-MLX	Native multimodal support, prefix caching for images
Embedding pool (4 nodes)	mlx-lm embedding server or sentence-transformers + MLX	No KV cache, no batching scheduler needed; encoder-only
Speech pool (2 nodes)	WhisperKit	Apple Neural Engine optimization, 0.46s real-time latency, 50% faster than non-MLX
Distributed / experimental (optional 3–4 nodes)	EXO	Only viable Mac Mini cluster framework; acceptable for off-peak 30B experiments
API gateway	LiteLLM Proxy (self-hosted)	Routing, budgets, rate limits, OIDC, admin UI
Load balancer	HAProxy	Mature, low-overhead, reliable health checks

Rejected: - LM Studio: desktop-only, no headless deployment - llama.cpp direct: no continuous batching, unsuitable for ≥ 5 concurrent users - Ollama: acceptable as fallback, not primary due to lack of paged KV cache

6. Distributed Inference Analysis

6.1 Distributed Inference on Apple Silicon: Mechanism

The only viable distributed inference strategy on this hardware is **pipeline parallelism**: split the model's transformer layers into contiguous groups, assign each group to a separate node, and pass activations sequentially between nodes.

Tensor parallelism (AllReduce across all heads per layer) is not viable because: - No NCCL equivalent for Metal/Apple Silicon - AllReduce across Ethernet adds $\sim 0.1\text{--}0.5$ ms per operation; a 70B model with 80 layers requires 160 AllReduce calls per forward pass = 16–80 ms of communication overhead per token - Total decode latency becomes communication-dominated, not compute-dominated

Model sharding (weight partitioning per layer) requires tensor parallelism and is excluded for the same reason.

Pipeline parallelism is viable because: - Activations passed between stages are small: a Llama 3.1 8B activation tensor is ~4 KB per token - At 4 KB activation, 1 Gbps Ethernet adds ~0.032 ms per hop — negligible compared to per-stage compute time - Each node computes its layer shard fully before passing to the next; no synchronization required during computation

6.2 Network Constraints

Interconnect	Latency (RTT)	Bandwidth	Suitability
1 Gbps Ethernet (LAN)	0.1–0.5 ms	125 MB/s	Acceptable for pipeline parallelism
10 Gbps Ethernet	0.05–0.1 ms	1.25 GB/s	Recommended for distributed pool
Thunderbolt 5	~0.01–0.05 ms	10 GB/s	Optimal; limited by Mac Mini port availability

Bandwidth math for activation transfer: - Llama 3.1 8B: activation = hidden_dim × batch_size × bytes = $4096 \times 1 \times 2 = 8$ KB per token - At 10 tok/s output: 80 KB/s inter-node — negligible even on 1 Gbps - **Conclusion:** Activation bandwidth is not the bottleneck. Latency and per-node compute time dominate.

6.3 KV Cache Behavior in Distributed Mode

In pipeline parallelism, each node caches only the KV pairs for its assigned layers. There is no cross-node KV cache synchronization.

- Node 0 (layers 0–15 of a 32-layer model): caches K/V for layers 0–15
- Node 1 (layers 16–31): caches K/V for layers 16–31
- KV cache per node = (total_KV_budget / 2) per-layer shard

This reduces the per-node KV cache vs. single-node hosting, allowing larger effective batch sizes when distributing. However, the sequential pipeline means that total throughput per request does not improve with node count — only aggregate multi-request throughput scales.

6.4 EXO Benchmark Data (Mac Mini Clusters)

Configuration	Model	Single-request tok/s	Multi-request aggregate	TTFT
1× M4 Pro 24GB	Llama 3.2 3B	49.3	49.3	—
2× M4 Pro 24GB	Llama 3.2 3B	44.4	95.7	—
3× M4 Pro 24GB	Llama 3.2 3B	39.7	108.8	—
8× M4 Pro 64GB	DeepSeek V3 671B	5.37	—	2.91s

Observation: Single-request latency degrades with node count (pipeline overhead). Multi-request aggregate throughput scales near-linearly. The pipeline approach is optimized for throughput, not latency.

Important caveat: The EXO benchmark nodes use M4 Pro 24 GB — not the 16 GB Mac Mini base variant. M4 Pro has 273 GB/s bandwidth vs M4's 120 GB/s. Extrapolating to 16 GB M4 base: expect single-request throughput approximately 40–50% lower.

6.5 Feasibility Matrix for Target Model Sizes

Assumption: Distributed pool uses 16 GB M4 nodes with 1 Gbps LAN.

Model	Q4 weight size	Nodes required	Single-request tok/s (est.)	Multi-req agg tok/s	TTFT (est.)	Verdict
Qwen 2.5 32B	~19 GB	3	6–10	18–30	3–6s	Marginal
Llama 3.1 30B (Llama 3.3)	~17 GB	3	7–12	21–36	2–5s	Marginal
Mixtral 8×7B	~26 GB	4	5–8	15–25	4–8s	Poor
Qwen 2.5 72B	~43 GB	6–7	3–5	9–15	6–12s	Impractical
Llama 3.1 70B	~40 GB	6–7	3–5	9–15	6–12s	Impractical

70B verdict: 7 nodes consumed, 5 tok/s per request, 12s TTFT. This is below interactive usability threshold (~10 tok/s for comfortable streaming). 17.5% of the entire cluster consumed by one pool. Not justified.

30B verdict: 3 nodes, 8 tok/s, 4s TTFT. Barely interactive. Consumes 7.5% of cluster. Acceptable for batch/async reasoning tasks (long-running agent loops where TTFT is not critical). **Off-peak only.**

6.6 Distributed Inference Operational Rules

1. Distributed pool is **not provisioned by default**. It is activated manually during off-peak hours via pool scheduler configuration.
2. Distributed pool is funded by borrowing nodes from the Fast pool (which has 2× headroom). During activation, 3–4 Fast pool nodes are drained and reassigned.
3. Requests routed to the distributed pool must have `X-Hydra-Pool: distributed` header — not accessible to default user groups.
4. No SLA is offered on distributed pool latency.
5. EXO, not vLLM-MLX, is the runtime for the distributed pool. EXO handles peer discovery and layer sharding automatically.

6.7 Why Tensor Parallelism Is Not Viable (Detailed)

For completeness, the specific blockers for tensor parallelism on this hardware:

Missing primitive 1: High-bandwidth collective operations. Tensor parallelism splits the attention and FFN layers across devices. After each linear projection, an AllReduce is required to sum partial results. NVIDIA systems use NVLink (600 GB/s for H100) + NCCL. Apple Silicon over Ethernet: 0.125 GB/s, 3,000–5,000× lower bandwidth. A single AllReduce across 4 nodes adds ~1 ms minimum; with 80 layers × 2 AllReduces = 160 ms communication per token at 70B scale. Completely dominates compute time.

Missing primitive 2: Peer-to-peer GPU memory. CUDA Unified Virtual Addressing allows GPU-to-GPU direct memory copies. Metal does not expose cross-device GPU memory access. All cross-node transfers go CPU → NIC → CPU, incurring PCIe round-trip on top of network latency.

Missing primitive 3: Process groups. PyTorch's `torch.distributed` with NCCL backend handles process group initialization, device mesh creation, and DTensor operations. The Metal/MPS backend does not implement `torch.distributed`. vLLM-MLX does not support tensor parallelism on Apple Silicon.

Conclusion: Tensor parallelism on this cluster is architecturally blocked. Pipeline parallelism via EXO is the only practical distributed inference option.

7. Quantization Strategy and Policy

7.1 GGUF Quantization Mechanics

GGUF K-quants (Q2_K through Q6_K) use block-wise quantization with separate scaling factors per super-block. The K suffix denotes a mixed-precision scheme where some tensors (embedding, output, certain attention weights) are stored at higher precision than the named bit-width. This produces better quality per bit than naive uniform quantization but adds slight overhead to file size and metadata.

Storage formula approximation:

$$\text{file_size_GB} \approx (\text{params_B} \times \text{bits_per_weight}) / 8$$

This is approximate; actual files are 5–15% larger due to: - Non-quantized tensors (embeddings, layer norms stored at Q8 or FP16) - GGUF metadata and tokenizer data - Scaling factor overhead in K-quants

7.2 Quantization Reference Table — 7B Parameter Model

Quant type	Bits/weight (effective)	File size	RAM (loaded)	Quality vs FP16	Primary constraint on 16 GB node	Use case
FP16	16	~14 GB	~15.3 GB	Baseline	Exceeds weight ceiling (~9.6 GB) — BLOCKED	Not used
Q8_0	8.0	~8.2 GB	~8.7 GB	~99%	Exceeds weight ceiling — BLOCKED for serving	Evaluation only
Q5_K_M	5.5	~5.5 GB	~6.0 GB	~98%	Comfortable, reduces KV headroom slightly	Quality-critical tasks
Q4_K_M	4.8	~4.8 GB	~5.2 GB	~97% (<1% bench delta)	Default. Optimal balance	Primary serving
Q3_K_M	3.4	~3.9 GB	~4.3 GB	~94%	Acceptable only for low-stakes tasks	Draft generation
Q2_K	2.6	~3.1 GB	~3.5 GB	~85%	Severe degradation on reasoning/coding	Blocked for use

Q8_0 note: Exceeds the 9.6 GB weight ceiling for 7B models, meaning a 7B at Q8_0 leaves only ~4.7 GB for KV cache — insufficient for concurrent serving. Q8_0 is permitted only for ≤3B models and offline evaluation.

Q4_K_M justification for primary fleet: Benchmark degradation is consistently below 1% on MMLU, HumanEval, and HellaSwag relative to FP16. Quality impact is not user-perceptible for the target use cases. KV cache headroom is maximized.

7.3 Quantization Reference Table — By Model Size Class

Model size	FP16 weight size	Q4_K_M weight size	Q4 fits 16 GB?	Recommended quant	KV cache left after weights
0.5B	~1 GB	~0.3 GB	Yes	Q8_0 (quality first)	~13.1 GB
1.5B	~3 GB	~1.0 GB	Yes	Q5_K_M or Q8_0	~12.4 GB
3B	~6 GB	~2.0 GB	Yes	Q5_K_M	~11.4 GB
7B	~14 GB	~4.5 GB	Yes	Q4_K_M	~8.9 GB
8B	~15 GB	~4.8 GB	Yes	Q4_K_M	~8.6 GB
12B	~24 GB	~7.6 GB	Yes (tight)	Q4_K_M only	~5.8 GB
14B	~28 GB	~9.0 GB	Yes (marginal)	Q4_K_M only	~4.4 GB
22B	~44 GB	~13 GB	No	Not single-node	—
30B	~60 GB	~17 GB	No	Distributed only	—
70B	~140 GB	~40 GB	No	Not feasible	—

7.4 Quantization Governance Policy

The following rules are enforced by the model registry. No inference node will load a model that violates these rules. The policy engine at the gateway rejects requests specifying non-approved variants.

```

RULE 1: FP16 BLOCKED for all models with params > 3B on inference nodes.
        Rationale: 7B FP16 = 14 GB, exceeds effective weight ceiling.

RULE 2: Q8_0 BLOCKED for serving on models > 3B.
        Rationale: 7B Q8_0 = 8.2 GB, leaves insufficient KV budget for multi-user
        serving.
        Q8_0 permitted for offline evaluation and embedding pool.

RULE 3: Q3_K_M BLOCKED for reasoning, coding, and planning task categories.
        Rationale: 6% quality degradation is unacceptable for structured output tasks.
        Q3_K_M permitted only for fast/chat category as explicit user override.

RULE 4: Q2_K BLOCKED for all production serving.
        Rationale: 15% quality degradation is unacceptable for any user-facing task.
        Q2_K available only in experimental pool with explicit admin authorization.

RULE 5: For 12B-14B models, Q4_K_M is the ONLY permitted serving quantization.
        Rationale: Tighter weight budget forces single-quant selection.

RULE 6: For ≥30B models, Q3_K_M or Q4_K_M is permitted for distributed-pool only.
        Rationale: Must minimize weights to fit across minimum node count.

RULE 7: Arbitrary model loading is blocked at the runtime level.
        Rationale: Only models listed in the registry with approved=true may be loaded.
        Node startup scripts verify model SHA256 against registry before serving.

```

7.5 MLX Quantization Format Note

vLLM-MLX uses MLX's internal quantization format, not GGUF. The MLX format supports 4-bit and 8-bit quantization via `mlx.core.quantize`. This is equivalent in quality to Q4_K_M and Q8_0 GGUF respectively, though the file format and bit-packing differ.

For the purposes of policy enforcement, MLX 4-bit ↔ Q4_K_M and MLX 8-bit ↔ Q8_0 are treated as equivalent tiers. The model registry stores both GGUF and MLX artifacts with the quantization tier recorded as the canonical attribute.

8. Node Pool Allocation and Model Tiering

8.1 Pool Design — 40 Inference Node Cluster

The 40-node cluster is divided as follows. Infrastructure nodes (HAProxy, gateway, model store, observability) are in addition and use 2-4 of the lowest-spec nodes (M1).

Pool	Nodes	Chip preference	Model(s)	Quant	Max context (enforced)	Concurrent/ node (operational)	Total pool capacity
Fast text	20	M4 first, M3 fill	Qwen 2.5 7B (primary), Llama 3.1 8B (alternate)	Q4_K_M	8,192 tokens	10–24	200–480 simultaneous
Reasoning	8	M4 first, M3 fill	Qwen 2.5 14B	Q4_K_M	4,096 tokens	3–7	24–56 simultaneous
Vision	6	M3 or M4	MiniCPM-V 2.6 (8B) primary, Qwen-VL 7B alternate	Q4_K_M	2,048 text + image	2–4	12–24 simultaneous
Embedding	4	M1 acceptable	BGE-M3 (569M)	FP16	8,192 tokens (encoder)	Batch 256+	Very high (1000s/s)
Speech	2	M1 acceptable	Whisper Large v3 Turbo (1B compressed, 0.6 GB)	Compressed	N/A — audio input	10+ concurrent streams	20+ streams

Note: Infrastructure overhead (HAProxy: 2 nodes, LiteLLM gateways: 2 nodes, Model store: 2 nodes, Observability: 1 node) uses 7 nodes. Total cluster including infrastructure: 47 nodes. If cluster is hard-capped at 40 inference-only nodes, infrastructure runs on separate hardware or in lightweight VMs/containers on M1 nodes shared with embedding/speech pools.

Recommended split for exactly 40 Mac Minis:

```

Fast text   : 20 nodes (M4/M3)
Reasoning   :  8 nodes (M4/M3)
Vision      :  6 nodes (M3/M4)
Embedding   :  3 nodes (M1)
Speech      :  1 node  (M1)
Infrastructure (HAProxy, GW, Store, Obs) : 2 nodes shared (M1)
-----
Total       : 40 nodes

```

Infrastructure components (LiteLLM, HAProxy, Prometheus, Redis) run as Docker containers on the 2 shared M1 infrastructure nodes. Resource usage is low enough that this does not impact inference on those nodes.

8.2 Chip-to-Pool Assignment Matrix

Node count	Chip	Assigned pool(s)	Rationale
14	M4	Fast text (10), Reasoning (4)	Highest bandwidth → latency-sensitive decode
6	M3	Fast text (4), Vision (2)	Mid-bandwidth; sufficient for primary workloads
6	M3	Reasoning (4), Vision (2)	Fill remaining reasoning + vision
12	M1	Embedding (3), Speech (1), Fast text overflow (6), Infrastructure (2)	Bandwidth-insensitive workloads

This assignment is illustrative. The actual chip distribution in the cluster is determined by inventory. The scheduler must be configured with per-node chip labels and pool assignments.

8.3 Model Tiering Strategy

Three model quality tiers are defined. Tier assignment determines which pool handles requests and what concurrency is available.

Tier 1 — High throughput, standard quality

- Target pool: Fast text
- Models: Qwen 2.5 7B Q4_K_M, Qwen 2.5 Coder 7B Q4_K_M, Llama 3.1 8B Q4_K_M
- Max context: 8,192 tokens
- Max output: 4,096 tokens
- Concurrent: 10–24 per node
- Use cases: chat, coding, summarization, RAG, execution

Tier 2 — Lower throughput, higher quality

- Target pool: Reasoning
- Models: Qwen 2.5 14B Q4_K_M, Mistral Nemo 12B Q4_K_M
- Max context: 4,096 tokens
- Max output: 2,048 tokens
- Concurrent: 3–7 per node
- Use cases: reasoning, planning, structured analysis, multi-step tasks

Tier 3 — Specialized modalities

- Target pools: Vision, Embedding, Speech (each is a separate specialized tier)
- Models: MiniCPM-V 2.6 (vision), BGE-M3 (embedding), Whisper Large v3 Turbo (speech)

- Concurrency varies by modality

8.4 Task Category to Pool Routing Map

Task category	Target pool	Default model	Alternate model	Context default	Notes
chat	fast	Qwen 2.5 7B Q4	Llama 3.1 8B Q4	4K	Highest volume workload
coding	fast	Qwen 2.5 Coder 7B Q4	DeepSeek Coder 6.7B Q4	8K	Larger context for repo-level context
summarization	fast	Llama 3.1 8B Q4	Qwen 2.5 7B Q4	4K	—
rag	fast	Llama 3.1 8B Q4	Qwen 2.5 7B Q4	4K	Context injected by retriever pre-request
execution	fast	Qwen 2.5 7B Q4	—	2K	Short prompt/output cycles
reasoning	reasoning	Qwen 2.5 14B Q4	Mistral Nemo 12B Q4	4K	Long CoT chains
planning	reasoning	Qwen 2.5 14B Q4	—	4K	Multi-step structured output
vision	vision	MiniCPM-V 2.6	Qwen-VL 7B Q4	2K + image	Image must be resized ≤1080p before routing
embedding	embedding	BGE-M3	BGE-Large-EN	8K (encoder)	Batch allowed, streaming not applicable
speech	speech	Whisper Large v3 Turbo	Whisper Medium	N/A	Audio input, text output
distributed	experimental	Qwen 2.5 32B Q4	—	4K	Admin-authorized only

Task classification mechanism:

Requests are tagged by one of three methods, in priority order: 1. Explicit `X-Hydra-Task` HTTP header from the client. 2. `task` field in the request body metadata. 3. LiteLLM gateway classifier: keyword-based routing rules applied to the system prompt. Rules are static and configured per deployment.

If no task tag is resolvable, the gateway defaults to `chat` and routes to the fast pool.

8.5 Model Loading and Node Startup

Each inference node runs a single model, loaded at startup. Model selection is determined by pool assignment, which is encoded in the node's systemd unit or Docker Compose service definition.

```
# Example: Fast pool node startup
vllm-mlx serve Qwen/Qwen2.5-7B-Instruct \
  --quantization mlx-4bit \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.90 \
  --max-num-seqs 24 \
  --port 8000 \
  --host 0.0.0.0
```

```
# Example: Reasoning pool node startup
vllm-mlx serve Qwen/Qwen2.5-14B-Instruct \
  --quantization mlx-4bit \
  --max-model-len 4096 \
  --gpu-memory-utilization 0.90 \
  --max-num-seqs 7 \
  --port 8000 \
  --host 0.0.0.0
```

The `--max-num-seqs` flag is the primary KV cache concurrency guard at the runtime level. Its value must be consistent with the KV budget calculations in Section 3.

Model weights are pulled from the MinIO model store on first boot and cached to local SSD. Subsequent restarts serve from local cache. SHA256 verification runs before serving begins.

9. Governance and Policy Enforcement

9.1 Governance Layers

Policy enforcement is defense-in-depth across three layers. No single layer is sufficient; each enforces different aspects of the policy:

Layer 1 – Gateway (LiteLLM):

- Authentication and group resolution
- Per-user and per-group token budget enforcement
- Rate limiting (requests per minute)
- Concurrent request limits
- Task category classification and ACL check
- Input token counting and context window rejection
- Model allowlist enforcement

Layer 2 – Router (within LiteLLM):

- Pool-level routing based on task category
- Node selection (least-busy)
- Fallback model selection on node failure

Layer 3 – Runtime (vLLM-MLX per node):

- `--max-num-seqs`: hard concurrency cap at runtime level
- `--max-model-len`: hard context window cap
- `--gpu-memory-utilization`: hard KV cache memory cap
- Refuses to load models not present in local registry cache

9.2 User Group Definitions

Four user groups are defined. Additional groups can be created by an administrator. Group membership is resolved from OIDC claims or API key metadata at request time.

Group	Description	Example members
<code>default</code>	General staff, no AI-specific role	All employees not in a specialized group
<code>engineering</code>	Software engineers, ML practitioners	Engineering org
<code>data_science</code>	Data scientists, analysts	DS/Analytics org
<code>admin</code>	Platform administrators	AI infra team

9.3 Governance Policy Schema (Full Reference)

```

# hydra-policy.yaml
# Loaded by LiteLLM Proxy at startup. Redis-backed counters for rate limits and budgets.
# All token counts refer to prompt + completion combined unless noted.

governance:

# -----
# USER GROUP POLICIES
# -----

groups:

  default:
    # Token budgets
    tokens_per_day: 100_000
    tokens_per_month: 2_000_000

    # Rate limiting
    requests_per_minute: 10
    requests_per_hour: 300

    # Concurrency
    max_concurrent_requests: 3

    # Access control
    allowed_pools:
      - fast
      - embedding
    allowed_models:
      - qwen-2.5-7b-instruct
      - llama-3.1-8b-instruct
      - bge-m3
    allow_vision: false
    allow_speech: false
    allow_reasoning_pool: false
    allow_distributed_pool: false

    # Context limits (tokens)
    max_context_tokens: 4_096
    max_generation_tokens: 2_048
    max_input_tokens: 2_048

    # Quantization override: cannot request specific quantization
    # Server enforces pool default
    allow_quantization_override: false

  engineering:
    tokens_per_day: 500_000
    tokens_per_month: 10_000_000

    requests_per_minute: 30
    requests_per_hour: 1_000

    max_concurrent_requests: 10

```

```

allowed_pools:
  - fast
  - reasoning
  - vision
  - embedding
  - speech
allowed_models: all # all registry-approved models
allow_vision: true
allow_speech: true
allow_reasoning_pool: true
allow_distributed_pool: false

max_context_tokens: 8_192
max_generation_tokens: 4_096
max_input_tokens: 4_096

allow_quantization_override: false

data_science:
  tokens_per_day: 300_000
  tokens_per_month: 6_000_000

  requests_per_minute: 20
  requests_per_hour: 600

  max_concurrent_requests: 8

  allowed_pools:
    - fast
    - reasoning
    - embedding
  allowed_models: all
  allow_vision: false
  allow_speech: false
  allow_reasoning_pool: true
  allow_distributed_pool: false

  max_context_tokens: 8_192
  max_generation_tokens: 4_096
  max_input_tokens: 4_096

  allow_quantization_override: false

admin:
  tokens_per_day: unlimited
  tokens_per_month: unlimited

  requests_per_minute: 60
  requests_per_hour: unlimited

  max_concurrent_requests: 20

  allowed_pools: all

```

```

    allowed_models: all
    allow_vision: true
    allow_speech: true
    allow_reasoning_pool: true
    allow_distributed_pool: true

    max_context_tokens: 16_384
    max_generation_tokens: 8_192
    max_input_tokens: 8_192

    allow_quantization_override: true

# -----
# MODEL GOVERNANCE
# -----
models:

# Explicit allowlist of loadable models (registry-approved=true required)
allowlist:
  - id: qwen-2.5-7b-instruct
    pool: fast
    quant: q4_k_m
    max_context: 8192
    sha256: <registry-value>

  - id: qwen-2.5-coder-7b-instruct
    pool: fast
    quant: q4_k_m
    max_context: 8192
    sha256: <registry-value>

  - id: llama-3.1-8b-instruct
    pool: fast
    quant: q4_k_m
    max_context: 8192
    sha256: <registry-value>

  - id: qwen-2.5-14b-instruct
    pool: reasoning
    quant: q4_k_m
    max_context: 4096
    sha256: <registry-value>

  - id: minicpm-v-2.6
    pool: vision
    quant: q4_k_m
    max_context: 2048
    sha256: <registry-value>

  - id: bge-m3
    pool: embedding
    quant: fp16
    max_context: 8192
    sha256: <registry-value>

```

```

- id: whisper-large-v3-turbo
  pool: speech
  quant: compressed
  sha256: <registry-value>

# Blocked behaviors
arbitrary_model_load: false           # Runtime refuses models not in allowlist
allow_model_pull_from_hf: false       # Nodes cannot pull from HuggingFace at runtime
allow_fp16_serving_above_3b: false
allow_q8_serving_above_3b: false
allow_q2_production: false
allow_q3_for_reasoning_tasks: false

# -----
# RESOURCE GOVERNANCE (per node)
# -----
nodes:
  memory_pressure_reject_threshold_pct: 85
  memory_pressure_drain_threshold_pct: 90
  max_kv_cache_gb:
    fast: 8.5
    reasoning: 4.4
    vision: 8.0
  embedding: 12.0    # no KV cache; this is total inference memory
  speech: 15.0       # model is tiny, nearly all RAM available
  health_check_interval_s: 10
  health_check_failure_threshold: 3    # 3 consecutive failures = remove from pool
  graceful_drain_timeout_s: 30
  model_sha256_verify_on_load: true
  allow_runtime_model_swap: false     # No hot-swapping models; requires restart

```

9.4 Enforcement Execution Path

The following decision tree is executed by the LiteLLM gateway on every incoming request. Reject actions return HTTP 429 (rate/quota) or HTTP 403 (access) or HTTP 400 (validation).

REQUEST RECEIVED

- └─ 1. AUTH: Validate API key or OIDC token
→ Fail: HTTP 401
- └─ 2. RESOLVE: Map key/token to user + group
→ Fail: HTTP 403
- └─ 3. RATE LIMIT: Check requests_per_minute counter (Redis)
→ Exceeded: HTTP 429 (retry-after header set)
- └─ 4. BUDGET: Check tokens_per_day counter (Redis)
→ Exceeded: HTTP 429
- └─ 5. CONCURRENCY: Check max_concurrent_requests (Redis active set)
→ Exceeded: HTTP 429
- └─ 6. TASK CLASSIFY: Determine task category from header / body / classifier
→ Default: 'chat'
- └─ 7. ACL CHECK: Verify group is allowed for target pool
→ Fail: HTTP 403
- └─ 8. MODEL CHECK: Verify requested model (if specified) is in group allowlist
→ Fail: HTTP 403
- └─ 9. TOKEN COUNT: Count input tokens using model tokenizer
(tiktoken for Llama/Mistral; Qwen tokenizer for Qwen models)
- └─ 10. CONTEXT VALIDATE: Check $\text{input_tokens} + \text{max_gen_tokens} \leq \text{group.max_context_tokens}$
AND $\leq \text{model.max_context}$
→ Fail: HTTP 400 (context_too_long)
- └─ 11. NODE SELECT: Query pool for least-busy available node
→ No nodes available: HTTP 503 (queue or return immediately per config)
- └─ 12. FORWARD: Proxy request to selected inference node
- └─ 13. POST-RESPONSE: Increment consumed_tokens counter (Redis), write audit log

9.5 Rate Limit Counter Design

Rate limit counters use Redis sorted-set sliding window implementation:

```
Key pattern: rate:<user_id>:<window>
Window options: 1min, 1hour, 1day, 1month
Expiry: set to window duration + 1 minute
Increment: +1 on request receipt
Check: ZCOUNT <key> <now - window_ms> <now> → compare against limit
```

Token budget counters:


```

Key pattern: budget:<user_id>:tokens:<period>
Period: day (reset at midnight UTC), month (reset on 1st)
Increment: +output_tokens after response completes
Check: GET <key> → compare against limit before request

```

9.6 Context Window Governance Architecture

Context window enforcement occurs at two points:

Point 1 — Gateway (pre-flight):

The gateway tokenizes the request before forwarding. This is exact tokenization using the target model's tokenizer, not an approximation.

```

# Pseudocode — executed by LiteLLM gateway middleware
tokenizer = load_tokenizer(target_model_id)
input_token_count = len(tokenizer.encode(full_prompt))
max_allowed_context = min(
    group_policy.max_context_tokens,
    model_policy.max_context
)
if input_token_count + request.max_tokens > max_allowed_context:
    return HTTPError(400, "context_too_long",
        detail=f"Input: {input_token_count} tokens. "
            f"Max allowed: {max_allowed_context - request.max_tokens}")

```

The tokenizer is loaded once at gateway startup and cached in memory. Tokenizer selection: - Llama 3.x: tiktoken (cl100k_base compatible, or llama3 tokenizer) - Qwen 2.5: Qwen's tokenizer (qwen.tiktoken or HuggingFace tokenizer) - Mistral: Mistral sentencepiece tokenizer

Point 2 — Runtime (enforcement):

vLLM-MLX enforces `--max-model-len` as an absolute runtime cap. Even if the gateway passes a request that would exceed the per-pool context limit, the runtime rejects it with a 400 error. This provides defense-in-depth.

Truncation policy:

The system does NOT perform automatic silent truncation. Truncation is non-deterministic and leads to malformed prompts from the model's perspective (e.g., cut-off system prompt, split code blocks). Behavior:

1. Gateway counts tokens.
2. If count exceeds limit, gateway REJECTS the request with a structured error response including the token count and the limit.
3. Client must reduce input. Gateway returns the exact token count so the client can adjust.

RAG context bounding:

For `rag` task category, the gateway applies additional enforcement: - Maximum retrieved context chunks: configurable per group (default: 5 chunks of 512 tokens each = 2,560 tokens max retrieval) - Total context budget = `input_tokens + retrieved_tokens + max_generation_tokens` $\leq$ `max_context` - If retrieval budget would overflow, the gateway truncates retrieved chunks (fewest-relevant chunks removed first, based on retriever score rank)

This is the only context truncation performed automatically — and only on retrieval chunks, never on the user prompt.

10. Multimodal Capability and Isolation

10.1 Supported Modality Pools

Modality	Models	Pool	Runtime	Isolation mechanism
Text	Qwen 2.5 7B, Llama 3.1 8B, Qwen 2.5 14B	Fast, Reasoning	vLLM-MLX	Dedicated node pools
Vision (image+text)	MiniCPM-V 2.6, Qwen-VL 7B	Vision	vLLM-MLX	Dedicated node pool, separate queue
Speech (audio→text)	Whisper Large v3 Turbo	Speech	WhisperKit	Dedicated node pool, audio preprocessing sidecar
Embedding	BGE-M3, BGE-Large-EN	Embedding	mlx-lm or sentence-transformers	Dedicated node pool, batch-optimized

10.2 Vision Model Isolation Design

Why isolation is required:

- Vision models are fundamentally different workloads from text models in memory profile and compute pattern.
- Image encoding (ViT forward pass) is compute-intensive and inflates time-to-first-token significantly.
- Without isolation, vision requests could saturate nodes intended for text serving, causing latency spikes for text users.
- Vision models occupy the same weight size class as text models (~8B parameters) but require additional memory for vision encoder weights (+500 MB to +2 GB depending on architecture).

MiniCPM-V 2.6 memory profile:

```
LLM backbone (8B): ~4.8 GB Q4_K_M
Vision encoder: ~500 MB
Total weights: ~5.3 GB
KV cache budget: ~8.1 GB
```

Image token efficiency:

MiniCPM-V 2.6 produces 640 tokens per image at 1.8 megapixel resolution. This is 75% fewer tokens than competing vision models (which generate 2,000–3,000 image tokens). This directly translates to lower KV cache consumption per vision request, enabling higher concurrency.

Model	Image tokens per 1.8MP image	KV increase per request	Impact
LLaVA 1.6 34B	2,880	+360 MB @ 8B scale	Very high
Qwen-VL 7B	~1,500–2,000	+188–250 MB	High
MiniCPM-V 2.6	640	+80 MB	Low

Image preprocessing pipeline:

Images are preprocessed before reaching the vision inference node. The gateway (or a sidecar preprocessing service on the vision node) performs:

- 1. Decode base64 or accept multipart form data
- 2. Validate image format (JPEG, PNG, WebP)
- 3. Resize to max 1,920×1,920 pixels (preserving aspect ratio) if larger
- 4. Re-encode as JPEG at 85% quality to reduce transmission size
- 5. Convert to model-specific format (pixel values tensor or base64 for API)
- 6. Forward to vision node with preprocessed image

Concurrency on vision pool:

```
Node: 6B vision model, Q4_K_M, ~5.3 GB weights
KV budget: ~8.1 GB
KV per request (2K text + 640 image tokens = 2,640 tokens × 0.125 MB/token):
  = 330 MB per request
Max concurrent: 8.1 GB / 330 MB ≈ 24 theoretical; 10–12 operational
```

However, image encoding on a 10-core Apple GPU takes 500ms–2s per image. Effective concurrency is limited by encoding throughput, not KV cache. Operational target: 4–6 concurrent vision requests per node.

10.3 Speech Model Isolation Design

Whisper Large v3 Turbo specifications:

Attribute	Value
Parameters	~1 billion (compressed from 1.6B)
File size	~0.6 GB (compressed by WhisperKit)
Memory footprint	~1.0–1.5 GB loaded
Word Error Rate	2.2% (within 1% of uncompressed)
Real-time latency	0.46 seconds
Acceleration	Apple Neural Engine (ANE) — near-peak utilization

Isolation rationale:

Speech workloads are fundamentally CPU/NPU-bound, not GPU-bound. Whisper uses the Neural Engine for encoder operations and the GPU/CPU for decoder. Loading Whisper on a text-serving node would compete for GPU memory with no benefit — the NPU is idle on text nodes anyway.

Speech node workload:

Each speech node runs WhisperKit. Two nodes provide 20+ simultaneous transcription streams. Each transcription takes 0.46s real-time for typical utterances. Node memory utilization: ~1.5 GB for model, leaving 11.9 GB available for processing audio buffers.

Audio input pipeline:

```
Client → Gateway → [Audio validation: format, duration ≤ 30min, size ≤ 100 MB]
    → Speech node (HTTP multipart or audio stream)
    → WhisperKit: decode → transcribe → return text
    → Gateway → Client
```

Supported audio formats: WAV, MP3, FLAC, M4A, OGG. Output: plain text or JSON with timestamps and word-level confidence.

Cross-modal chaining:

For workflows that require speech-to-text followed by LLM processing:

```

Client sends: audio file + LLM system prompt
Gateway pipeline:
  1. Route audio to speech pool → get transcript text
  2. Compose: system_prompt + "Transcript: " + transcript_text
  3. Route composed prompt to fast/reasoning pool → get LLM response
  4. Return LLM response to client

```

This chaining is implemented at the gateway level. The client sends a single multimodal request; the gateway handles the pipeline internally.

10.4 Embedding Pool Design

BGE-M3 specifications:

Attribute	Value
Parameters	569M
Architecture	XLM-RoBERTa-Large (encoder-only)
File size FP16	~1.1 GB
Max input length	8,192 tokens
Output	Dense vectors (1024-dim), sparse lexical vectors, ColBERT multi-vectors
Languages	100+

No KV cache:

Embedding models are encoder-only. They do not use autoregressive decoding and have no KV cache. Memory consumption does not grow with input length — the model processes the full input in one forward pass.

Batching:

Embedding requests should be batched at the gateway level. Single-item embedding requests are inefficient. The gateway collects embedding requests for 50–100ms and dispatches them as batches of up to 256 items to the embedding pool.

```

Batch dispatch policy:
- Trigger: batch_size ≥ 64 OR wait_time ≥ 50ms
- Max batch: 256 items
- Priority items (explicit urgent flag): bypass batching

```

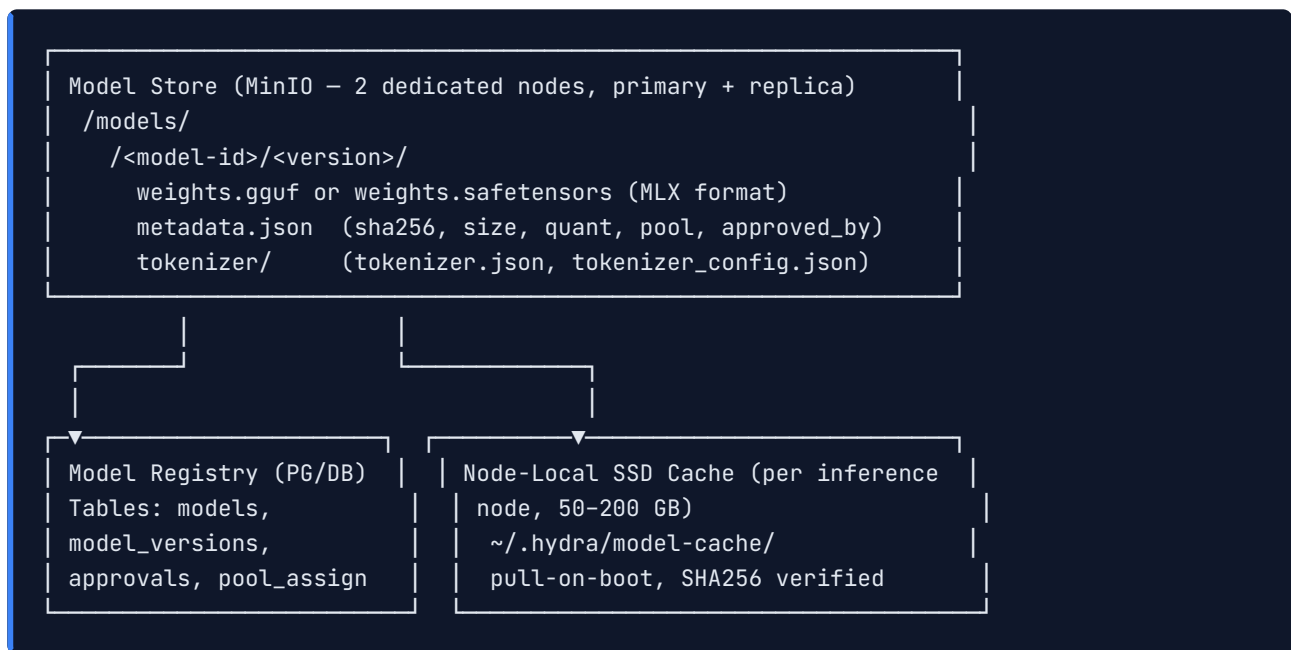
At batch size 256 on M4, BGE-M3 processes ~1,000–2,000 embedding requests per second per node. With 4 nodes, the embedding pool sustains 4,000–8,000 embeddings/second — sufficient for real-time RAG at any expected load.

Isolation from text pools:

Embedding requests **MUST** be routed to the embedding pool. They **MUST NOT** be served by text models (which would produce last-hidden-state embeddings of unpredictable quality). The task category `embedding` is hard-locked to the embedding pool at the routing layer.

11. Storage Layer

11.1 Storage Architecture Overview



11.2 MinIO Model Store

Deployment: Two dedicated Mac Mini nodes (M1 acceptable). Node 1 is primary; Node 2 is replica with MinIO's built-in replication.

Bucket structure:

```
s3://hydra-models/
  llama-3.1-8b-instruct/
    v1.0/
      llama-3.1-8b-instruct-q4_k_m.gguf      (4.8 GB)
      llama-3.1-8b-instruct-q4-mlx/         (MLX shards)
      metadata.json
      tokenizer/
  qwen-2.5-7b-instruct/
    v1.0/
      ...
  qwen-2.5-14b-instruct/
    v1.0/
      ...
  minicpm-v-2.6/
    v1.0/
      ...
```

Access policy: Inference nodes have read-only MinIO access credentials. Only the admin group (human operators) and CI/CD pipeline have write access. Credentials are distributed via environment variables, never stored in config files.

Sizing: Total storage required for approved model set:

Model	Format	Size
Qwen 2.5 7B Q4_K_M	GGUF + MLX	~5 GB
Qwen 2.5 Coder 7B Q4_K_M	GGUF + MLX	~5 GB
Llama 3.1 8B Q4_K_M	GGUF + MLX	~5 GB
Qwen 2.5 14B Q4_K_M	GGUF + MLX	~9 GB
MiniCPM-V 2.6 Q4_K_M	GGUF + MLX	~5.5 GB
BGE-M3 FP16	SafeTensors	~1.1 GB
Whisper Large v3 Turbo	Compressed	~0.6 GB
Subtotal (1 copy)		~31 GB
With replica		~62 GB

Two 256 GB SSD nodes provide ample capacity for the approved model set plus 4–5 additional candidate models.

11.3 Model Registry Database

Implementation: PostgreSQL 16 on the primary infrastructure node. Read replica on the secondary for HA reads.

Schema:

```

CREATE TABLE models (
    model_id      TEXT PRIMARY KEY,
    display_name  TEXT NOT NULL,
    family        TEXT NOT NULL,           -- llama, qwen, mistral, etc.
    params_b      FLOAT NOT NULL,
    architecture  TEXT NOT NULL
);

CREATE TABLE model_versions (
    version_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id      TEXT REFERENCES models(model_id),
    version       TEXT NOT NULL,
    quant_type    TEXT NOT NULL,           -- q4_k_m, q5_k_m, fp16, etc.
    quant_tier    TEXT NOT NULL,           -- tier enum: q4, q5, q8, fp16
    format        TEXT NOT NULL,           -- gguf, mlx, safetensors
    sha256        TEXT NOT NULL,
    size_bytes    BIGINT NOT NULL,
    store_path    TEXT NOT NULL,           -- MinIO object path
    pool_assignment TEXT NOT NULL,         -- fast, reasoning, vision, etc.
    max_context   INT NOT NULL,
    approved      BOOLEAN DEFAULT FALSE,
    approved_by   TEXT,
    approved_at   TIMESTAMPTZ,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(model_id, version, quant_type, format)
);

CREATE TABLE node_model_cache (
    node_id       TEXT NOT NULL,
    version_id    UUID REFERENCES model_versions(version_id),
    cached_at    TIMESTAMPTZ,
    sha256_verified BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (node_id, version_id)
);

CREATE TABLE model_approvals (
    approval_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    version_id    UUID REFERENCES model_versions(version_id),
    requested_by  TEXT NOT NULL,
    approved_by   TEXT,
    status        TEXT DEFAULT 'pending',  -- pending, approved, rejected
    notes         TEXT,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    resolved_at   TIMESTAMPTZ
);

```


11.4 Node-Local Model Cache

Each inference node maintains a local disk cache at `~/hydra/model-cache/`. On node boot, the startup script:

1. Reads the assigned model version from local configuration (systemd env file or Docker Compose env).
2. Checks if the model version (by version_id) is present in local cache.
3. If present: verifies SHA256. If match → proceed to serve. If mismatch → re-download.
4. If absent: downloads from MinIO model store.
5. Records `sha256_verified=true` in registry.
6. Starts vLLM-MLX server.

Cache invalidation: Triggered by version bump in node assignment. Node pulls new version on next restart. Old version is deleted after successful verification of new version.

Cache size per node: 50–200 GB NVMe SSD. A single 7B Q4 model (GGUF + MLX shards) occupies ~10–12 GB including tokenizer. Reserve 2× model size for concurrent download + serving during transitions.

11.5 Model Approval Workflow

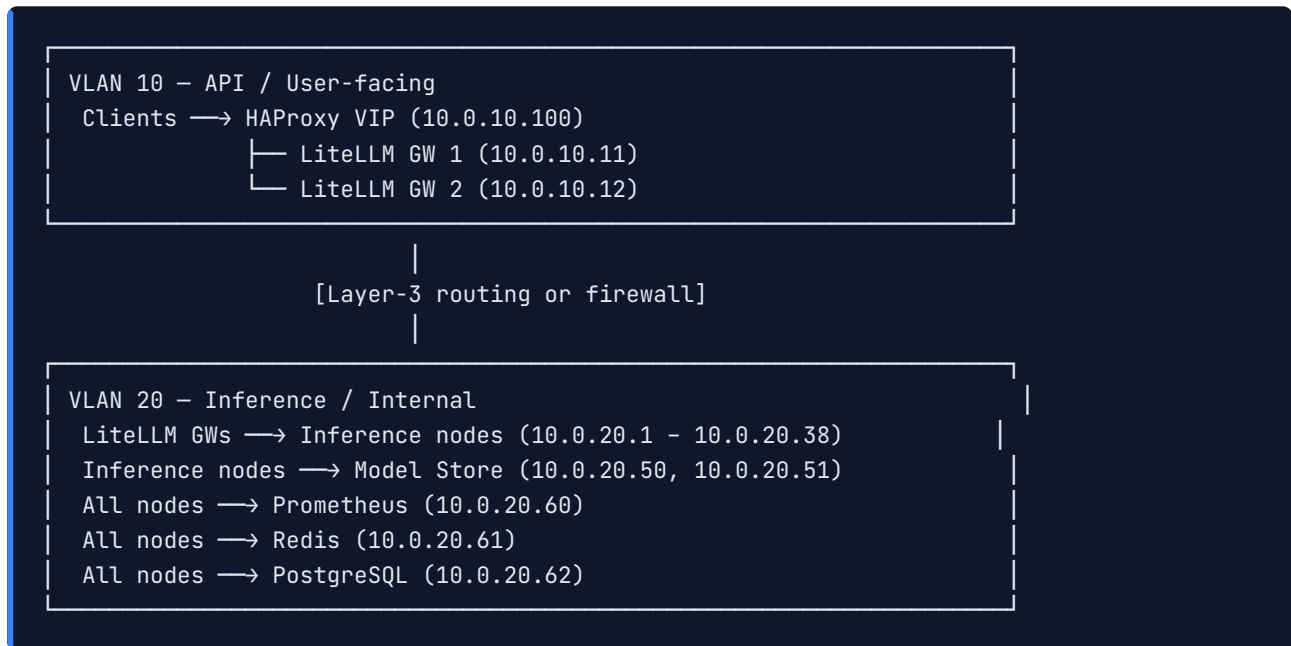
New models may not be deployed to inference nodes without registry approval. The workflow:

1. Operator uploads model artifact to MinIO (staging bucket)
2. Operator creates model_versions record with approved=false
3. Operator submits model_approvals request
4. Second operator reviews: checks license, security scan, quantization tier compliance
5. Approver sets approved=true on model_versions record
6. Model becomes eligible for pool assignment
7. Node restart (rolling) pulls new model version

No automated CI can approve models. Both upload and approval require authenticated admin operations.

12. Networking Model

12.1 Network Topology



VLAN 10 (API VLAN): Reachable from corporate network. Only ports 443 (HTTPS) and 80 (redirect to 443) exposed on the HAProxy VIP. No direct access to inference nodes from VLAN 10.

VLAN 20 (Inference VLAN): Isolated from direct user access. Inference nodes expose port 8000 (vLLM-MLX API) and port 9090 (Prometheus metrics) on VLAN 20 only. Gateways on VLAN 10 also have a VLAN 20 interface.

Firewall rules (VLAN 10 → VLAN 20): Only LiteLLM gateway MAC/IP pairs are permitted to initiate connections to inference node port 8000. All other VLAN 10 → VLAN 20 traffic is blocked.

Firewall rules (VLAN 20 → external): Inference nodes have NO outbound internet access. Model downloads happen only via MinIO (VLAN 20). This prevents runtime model pulls from HuggingFace or any external source.

12.2 Required Network Bandwidth

Traffic type	Source → Destination	Peak bandwidth	Notes
API requests	Clients → HAProxy	~50 Mbps	Text JSON payloads, small
API responses	Nodes → Clients (via GW)	~200 Mbps	Streaming token output
Vision input	Clients → Vision nodes	~500 Mbps	Images up to 5 MB each, 100/s peak
Audio input	Clients → Speech nodes	~100 Mbps	Audio files up to 100 MB
Model fetch	Nodes → MinIO	~5 Gbps burst	One-time per model version; 5 GB model at 1 Gbps = 40s
Metrics	Nodes → Prometheus	~10 Mbps	15s scrape interval, all 40 nodes
Distributed pool	Node → Node	Negligible (<1 Mbps)	4 KB activations per token

1 Gbps per node is sufficient for all primary workloads. Model fetch is the peak bandwidth event; staggered node restarts prevent simultaneous 40-node fetches.

10 Gbps recommendation: Apply to the distributed pool node pair only, if that feature is activated.

12.3 Service Discovery

mDNS (default): macOS supports Bonjour/mDNS natively. Each inference node advertises:

```
_hydra-inference._tcp.local.  
TXT: pool=fast model=qwen-2.5-7b version=1.0 chip=m4 status=healthy
```

LiteLLM gateway maintains a local pool table updated via mDNS browsing. Node failures cause mDNS deregistration → automatic removal from active pool within one health check cycle (≤30s).

Consul (alternative): For larger deployments or stricter control, replace mDNS with Consul agent on each node. Provides stronger health check configuration, KV store for dynamic pool configuration, and ACL-controlled node registration.

12.4 TLS and Certificate Management

- **External (VLAN 10):** HAProxy terminates TLS. Certificate from internal CA (recommended) or Let's Encrypt (if cluster has controlled external access). Certificate renewal automated via certbot or ACME client.
- **Internal (VLAN 20):** Plain HTTP between gateway and inference nodes by default (traffic is VLAN-isolated). Optional mTLS available via mutual certificate authentication if compliance requires encrypted internal transport.
- **Model store (MinIO):** HTTPS with internal CA certificate. MinIO enforces TLS for all connections.

12.5 Port Reference

Port	Protocol	Service	Network
443	HTTPS	HAProxy → LiteLLM GWs	VLAN 10 (external)
80	HTTP	HAProxy → redirect	VLAN 10
4000	HTTP	LiteLLM Proxy API	VLAN 20 (internal only)
8000	HTTP	vLLM-MLX inference nodes	VLAN 20 (internal only)
9090	HTTP	Prometheus metrics (all nodes)	VLAN 20
9100	HTTP	node_exporter (all Mac Minis)	VLAN 20
3000	HTTP	Grafana	VLAN 20 (admin access only)
9093	HTTP	Alertmanager	VLAN 20
6379	TCP	Redis	VLAN 20 (GW only)
5432	TCP	PostgreSQL	VLAN 20 (GW + admin)
9000	HTTPS	MinIO API	VLAN 20

13. Failure Handling and Recovery

13.1 Failure Mode Matrix

Failure mode	Detection mechanism	Detection time	Response	Recovery
Inference node unresponsive	LiteLLM health check: HTTP GET /health every 10s, 3 consecutive failures	~30s	Remove from active pool, reroute queued requests to other nodes	Node auto-re-registers via mDNS on recovery; returns to pool after 3 consecutive health check successes
Inference node memory OOM	node_exporter metric: memory_used > 90% threshold	~15s (metric lag)	Gateway stops routing new requests to node; in-flight requests complete or timeout	Automatic: pressure drops after in-flight requests complete; node re-admitted to pool
vLLM-MLX process crash	Health check failure (port 8000 unreachable)	~30s	Remove from pool	systemd / launchd restarts vLLM-MLX process; node re-serves after startup + model load (~30–60s)
LiteLLM Gateway failure	HAProxy health check: HTTP GET /health every 5s, 3 failures	~15s	HAProxy removes gateway from backend, routes to remaining gateway	Manual restart or auto-restart via Docker Compose restart policy
Both gateways fail	HAProxy has no healthy backend	Immediate	Clients receive HTTP 503	Start cold-spare gateway instance; ~60s manual recovery
HAProxy failure	keepalived detects primary failure (heartbeat timeout 3s)	~3s	keepalived promotes standby; VIP migrates to standby	HAProxy auto-restart; VIP migrates back after primary recovers
	Inference node startup fails to pull model	On boot	Nodes with cached local copy continue serving;	Restore MinIO; nodes

Failure mode	Detection mechanism	Detection time	Response	Recovery
Model store (MinIO) unavailable			nodes without cache cannot start	without local cache restart after MinIO recovery
Model store unavailable (mid-serving)	N/A — model is in node memory	N/A	No impact to active serving; only affects new node starts	Restore MinIO before scaling pool
Redis failure	LiteLLM cannot connect to Redis	Immediate	Gateway falls back to in-memory rate limiting (non-shared); dual-gateway rate limit accuracy degrades	Redis restart; counters reset (conservative: budgets may be under-counted for one window)
PostgreSQL failure	LiteLLM cannot verify model registry	On startup	Gateways serve from in-memory model list; new model approvals blocked	PG restore or failover to read replica; write operations resume after primary recovery
Network partition (nodes isolated from GW)	Health check failures from gateway side	~30s	Isolated nodes removed from pool	Partition resolves: nodes re-register; traffic resumes
Thermal throttling on node	Latency spike + reduced tok/s (Grafana alert)	~60s (inference latency metric)	No automatic action; monitoring alert triggers	Operator reduces pool allocation for affected node; hardware inspection
Model file corruption (SHA256 mismatch)	Pre-serve SHA256 check	On node startup	Node refuses to start; alert fires	Re-download from MinIO; SHA256 re-verified

13.2 Request-Level Failure Handling

Retry policy (gateway → inference node):

```

Attempt 1: Route to selected node
  → Failure (timeout 120s or HTTP 5xx): log, select next node in pool
Attempt 2: Route to alternate node
  → Failure: return HTTP 503 to client with Retry-After header
No further retries (prevents cascade amplification)

```

Timeout values:

Phase	Timeout
Connection to inference node	5s
Time to first token (TTFT)	30s
Full generation (streaming)	120s
Full generation (non-streaming)	300s
Image preprocessing	15s
Audio transcription	60s

In-flight request behavior during node drain:

When a node is removed from the pool (failure or graceful shutdown), in-flight requests continue until completion or timeout. The node receives a SIGTERM with 30-second grace period. After 30 seconds, SIGKILL is sent. Incomplete streaming requests receive a `stream_error` event; clients must retry.

13.3 Graceful Shutdown Procedure

Executed for planned maintenance (model updates, OS patches):

1. Operator marks node as draining via API:
PUT /v1/admin/nodes/{node_id}/state = draining
2. LiteLLM gateway stops routing NEW requests to node.
In-flight requests continue.
3. Wait for active request count on node to reach 0,
OR drain_timeout (30s) expires – whichever first.
4. Node sends SIGTERM to vLLM-MLX process.
vLLM-MLX completes current batch step, stops accepting new work.
5. After vLLM-MLX exits, node is safe for maintenance.
6. Post-maintenance: restart vLLM-MLX.
Node passes 3 consecutive health checks.
Operator marks node as active.
Traffic resumes.

13.4 Pool-Level Capacity Degradation Thresholds

If node failures reduce pool capacity, the following escalation policy applies:

Pool	Total nodes	Degraded (threshold)	Critical (threshold)	Action at critical
Fast text	20	<16 (80%)	<12 (60%)	Activate M1 overflow nodes; alert ops
Reasoning	8	<6 (75%)	<4 (50%)	Degrade reasoning requests to fast pool
Vision	6	<4 (67%)	<3 (50%)	Queue vision requests; SLA suspended
Embedding	4	<3 (75%)	<2 (50%)	Reduce embedding batch concurrency
Speech	2	1 (50%)	0 (0%)	Speech pool offline; alert

Reasoning pool degradation fallback: If reasoning pool drops to critical, the gateway routes `reasoning` and `planning` requests to the fast pool with an informational header `X-Hydra-Fallback: capacity`. Quality may be lower (7B vs 14B model). This is acceptable for continuity.

14. Security Controls

14.1 Authentication and Authorization

Authentication methods:

Method	Use case	Token type
OIDC SSO (corporate IdP)	Interactive users (web UI, Cursor IDE)	JWT bearer token, 1h expiry
API key	Programmatic access (scripts, CI/CD, agents)	Static key, 256-bit random, SHA256 stored
Service account	Internal services (e.g., RAG pipeline)	API key with restricted group

Group resolution: JWT claims include `groups` array. API keys map to a group via registry lookup. At every request, the group is resolved and governance policy applied. There is no anonymous access.

Authorization model:

```
Authorization = f(user_group, target_pool, requested_model, task_category)

Pool ACL: group.allowed_pools ∋ target_pool
Model ACL: group.allowed_models = 'all' OR requested_model ∈ group.allowed_models
Task ACL: derived from pool ACL (e.g., reasoning pool = reasoning tasks)
```

Enforcement is at the LiteLLM gateway layer. The inference nodes do NOT perform authentication — they trust requests from the gateway IP range only (enforced by VLAN 20 firewall rules).

14.2 Network Security

Inference node isolation:

- Inference nodes have no route to VLAN 10 (user network).
- Inference nodes have no outbound internet route. DNS resolution is internal only.
- Only ports 8000 (inference) and 9090 (metrics) are open on VLAN 20.
- SSH access to inference nodes is via jump host (bastion on VLAN 20). Public key authentication only. Password authentication disabled.

Gateway exposure:

- Port 443 only is exposed externally. HAProxy drops all other connections.
- TLS 1.3 minimum. TLS 1.0/1.1 disabled.

- HSTS header enforced.

Model store:

- MinIO requires TLS for all connections.
- Inference nodes authenticate with read-only access credentials (scoped access key per pool).
- Write access requires separate admin credentials.

14.3 Model Integrity

Every model artifact has a SHA256 hash recorded in the registry at upload time. Before an inference node serves any model, it runs:

```
sha256sum ~/.hydra/model-cache/<model_version_id>/<weights_file>  
# compare against registry value via admin API call  
# if mismatch: abort startup, fire alert, do not serve
```

This prevents: - Corrupted downloads (network errors during pull) - Tampered artifacts (adversarial modification of model store) - Accidental wrong-model serving (operator error in file path)

SHA256 is re-verified on every node restart, not only on first download.

14.4 Audit Logging

Every request processed by the gateway produces a structured JSON audit record:

```
{
  "event": "inference_request",
  "timestamp": "2026-03-06T14:23:11.412Z",
  "request_id": "req_7f3a2c1d...",
  "user_id": "alice@company.com",
  "group": "engineering",
  "task_category": "coding",
  "pool": "fast",
  "model": "qwen-2.5-coder-7b-instruct",
  "model_version": "v1.0",
  "node_id": "fast-node-07",
  "input_tokens": 1842,
  "output_tokens": 312,
  "total_tokens": 2154,
  "latency_ttft_ms": 892,
  "latency_total_ms": 4210,
  "status": "success",
  "http_status": 200,
  "client_ip": "10.0.10.45",
  "context_limit_enforced": false,
  "rate_limit_hit": false,
  "budget_remaining_today": 472846
}
```

Governance event records (separate stream):

```
{
  "event": "governance_action",
  "timestamp": "...",
  "action": "rate_limit_reject",
  "user_id": "bob@company.com",
  "group": "default",
  "limit_type": "requests_per_minute",
  "limit_value": 10,
  "current_count": 11,
  "request_id": "req_8b2c..."
}
```

Retention: 90 days minimum, immutable (append-only log storage). Stored to a separate log volume outside MinIO model storage.

Access: Audit logs readable only by admin group. Gateway service account writes only (no read permission).

14.5 Secrets Management

Secret type	Storage	Rotation
MinIO access keys (nodes)	Environment variable in systemd unit file (0600 permissions)	Manual, quarterly
PostgreSQL credentials	Environment variable in Docker Compose / systemd	Manual, quarterly
Redis AUTH password	Environment variable	Manual, quarterly
OIDC client secret (LiteLLM)	Environment variable	On IdP rotation
API keys (users)	PostgreSQL, SHA256 hash only	User-initiated or admin-forced
TLS private keys	/etc/ssl/ (0600, root only)	On certificate renewal

Credentials MUST NOT be stored in: - Config files committed to version control - Docker image layers - Grafana dashboard JSON - Log output

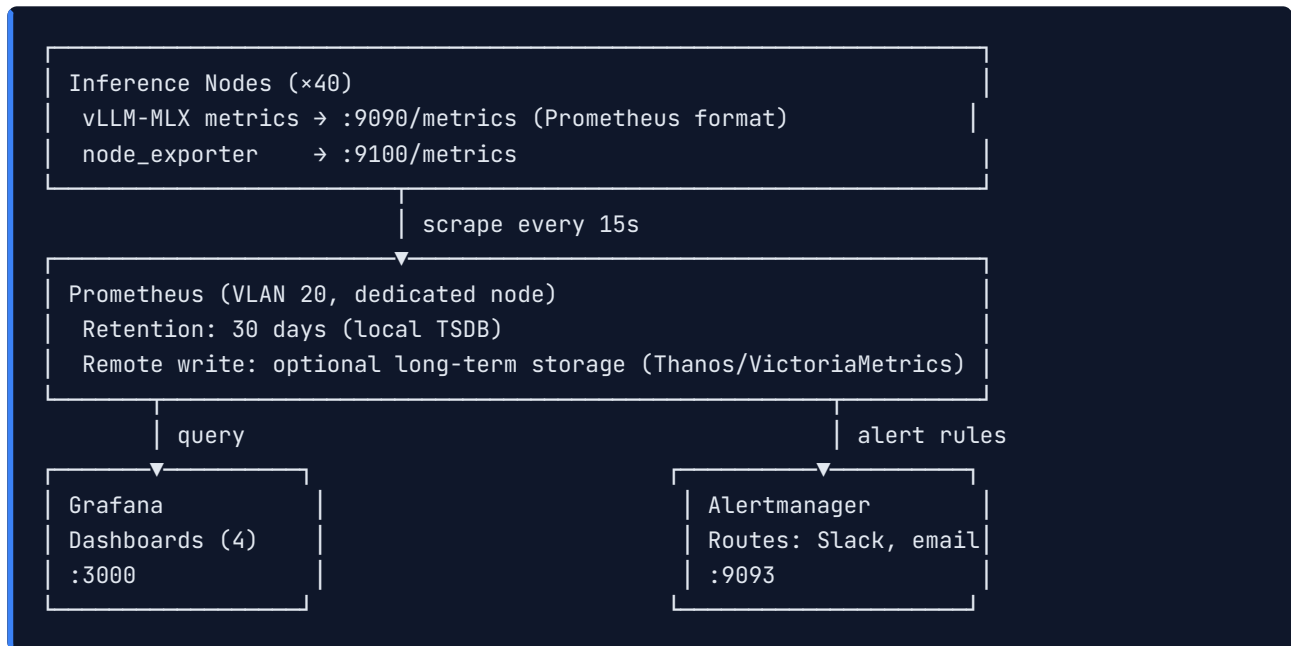
If HashiCorp Vault is available in the organization, integrate LiteLLM's Vault secret backend for all gateway credentials.

14.6 Software Supply Chain

- Inference runtimes (vLLM-MLX, WhisperKit) installed from verified sources only (GitHub releases with SHA256 verification or PyPI with hash pinning in requirements.txt).
- macOS software updates applied on a monthly schedule during maintenance windows.
- Model artifacts sourced from approved vendors only (HuggingFace repos listed in organization allowlist). No user-initiated model pulls.
- Dependency scanning (pip-audit or equivalent) run on gateway service container images before deployment.

15. Monitoring and Observability

15.1 Observability Stack Architecture



Additionally: - **LiteLLM gateway** exposes Prometheus-format metrics at `:4000/metrics` covering request rates, governance events, budget consumption, and routing decisions. - **HAProxy** exposes stats socket and Prometheus exporter at `:9104`. - **Redis** metrics via `redis_exporter` at `:9121`. - **PostgreSQL** metrics via `postgres_exporter` at `:9187`.

15.2 Complete Metrics Reference

Inference node metrics (vLLM-MLX):

Metric	Type	Labels	Description
<code>vllm_mlx_tokens_generated_total</code>	Counter	model, pool, node_id	Cumulative generated tokens
<code>vllm_mlx_tokens_prompt_total</code>	Counter	model, pool, node_id	Cumulative prompt tokens
<code>vllm_mlx_requests_total</code>	Counter	model, pool, node_id, status	Total requests by status
<code>vllm_mlx_request_duration_seconds</code>	Histogram	model, pool	End-to-end request duration
<code>vllm_mlx_time_to_first_token_seconds</code>	Histogram	model, pool	Prefill latency (TTFT)
<code>vllm_mlx_inter_token_latency_seconds</code>	Histogram	model, pool	Decode step latency
<code>vllm_mlx_num_requests_running</code>	Gauge	model, node_id	Active batched requests
<code>vllm_mlx_num_requests_waiting</code>	Gauge	model, node_id	Queued requests
<code>vllm_mlx_kv_cache_usage_percent</code>	Gauge	model, node_id	KV cache utilization %
<code>vllm_mlx_batch_size</code>	Gauge	model, node_id	Current batch size
<code>vllm_mlx_gpu_cache_hit_rate</code>	Gauge	model, node_id	Prefix cache hit rate

System metrics (node_exporter):

Metric	Description
<code>node_memory_MemAvailable_bytes</code>	Available unified memory
<code>node_memory_MemTotal_bytes</code>	Total unified memory
<code>node_cpu_seconds_total</code>	CPU utilization by mode
<code>node_disk_read_bytes_total</code>	SSD read throughput
<code>node_disk_written_bytes_total</code>	SSD write throughput
<code>node_network_receive_bytes_total</code>	Network RX per interface
<code>node_network_transmit_bytes_total</code>	Network TX per interface
<code>node_load1</code> / <code>node_load5</code>	System load averages

Gateway metrics (LiteLLM):

Metric	Type	Labels	Description
<code>litellm_requests_total</code>	Counter	user_group, pool, model, status	All request outcomes
<code>litellm_input_tokens_total</code>	Counter	user_group, pool, model	Prompt tokens consumed
<code>litellm_output_tokens_total</code>	Counter	user_group, pool, model	Generated tokens consumed
<code>litellm_rate_limit_hits_total</code>	Counter	user_group, limit_type	Rate limit enforcement
<code>litellm_budget_exhausted_total</code>	Counter	user_group	Budget exhaustion events
<code>litellm_context_reject_total</code>	Counter	user_group, pool	Context-too-long rejections
<code>litellm_routing_latency_seconds</code>	Histogram	pool	Gateway routing overhead
<code>litellm_pool_queue_depth</code>	Gauge	pool	Requests waiting for node
<code>litellm_node_active_requests</code>	Gauge	node_id, pool	Per-node active request count

15.3 Grafana Dashboard Specifications

Dashboard 1 — Cluster Overview:

Panels: - Cluster-wide tokens/second (sum over all nodes) — time series - Total requests/minute — time series - Active requests by pool (stacked) — time series - Node health grid (40 nodes, green/yellow/red status) — stat grid - P95 inference latency by pool — time series - KV cache utilization by pool (heatmap) — heatmap - Memory pressure alerts — alert list

Dashboard 2 — Pool Detail:

Parameterized by pool. Panels: - Tokens/second per node in pool — time series - Concurrent requests per node — time series - TTFT p50/p95/p99 — time series - Batch size distribution — histogram - KV cache usage per node — gauge - Queue depth — time series - Request success/failure/reject rate — time series

Dashboard 3 — User and Group Usage:

Panels: - Top 20 users by token consumption (today) — bar chart - Token consumption by group — pie + time series - Rate limit hits by group — time series - Budget utilization (% of daily limit, by group) — bar chart - Context rejection rate by group — time series - Requests by task category — pie

Dashboard 4 — Capacity Planning:

Panels: - 7-day and 30-day token consumption trend by pool — time series with forecast - Estimated days until capacity exhaustion at current growth rate — stat - Node utilization distribution (P95 across pool) — heatmap - KV cache peak utilization trend — time series - Queue depth trend — time series - Proposed node reallocation suggestions (manual annotation panel)

15.4 Alert Rules

All alerts route to Alertmanager. Critical alerts page on-call immediately; warning alerts send to Slack.

Alert name	Severity	Condition	Action
<code>NodeDown</code>	critical	Node health check fails for >30s	Page on-call; node auto-removed from pool
<code>PoolCapacityCritical</code>	critical	Pool active nodes < 50% of total	Page on-call
<code>MemoryPressureHigh</code>	warning	Node <code>MemAvailable</code> < 2 GB	Slack alert; reduce routing to node
<code>MemoryPressureCritical</code>	critical	Node <code>MemAvailable</code> < 1 GB	Page on-call; node draining initiated
<code>InferenceLatencyHigh</code>	warning	Pool p95 TTFT > 10s (5min window)	Slack alert
<code>InferenceLatencyCritical</code>	critical	Pool p95 TTFT > 30s (5min window)	Page on-call
<code>KVCacheNearFull</code>	warning	KV cache utilization > 85% sustained 5min	Slack alert
<code>QueueDepthHigh</code>	warning	Pool queue depth > 50 requests	Slack alert
<code>QueueDepthCritical</code>	critical	Pool queue depth > 200 requests	Page on-call
<code>GatewayDown</code>	critical	LiteLLM health check fails	HAProxy failover + page
<code>BudgetGroupExhausted</code>	warning	Group token budget > 90% of daily limit	Slack alert to group manager
<code>ModelLoadFailed</code>	critical	Node startup SHA256 mismatch or load error	Page on-call
<code>ModelStoreUnreachable</code>	critical	MinIO health check fails >60s	Page on-call
<code>RedisDown</code>	warning	Redis connection refused	Slack alert; rate limiting degrades

15.5 Log Management

Structured log format: JSON on all components. No unstructured text logs from application layer.

Log destinations:

Source	Destination	Retention
Audit logs (gateway)	Append-only log volume (local or NAS)	90 days minimum, immutable
Governance events (gateway)	Same as audit logs	90 days
Application logs (vLLM-MLX, gateway)	Local file + optional syslog forwarder	30 days
System logs (macOS unified log)	macOS log → optional forward	14 days
Prometheus TSDB	Local node	30 days (raw); longer via remote write

Log shipping: Optional Fluent Bit agent on each node forwards structured logs to central log aggregator (e.g., Loki for Grafana integration, or Elasticsearch).

16. Throughput Expectations

16.1 Per-Node Throughput Baselines

Baselines measured or extrapolated from published benchmarks. M4 base (120 GB/s) used for primary estimates.

Workload	Node chip	Model	Quant	Single-request tok/s	Concurrent	Aggregate tok/s
Fast text	M4	Qwen 2.5 7B	Q4_K_M	28–35	24	250–450
Fast text	M3	Qwen 2.5 7B	Q4_K_M	22–28	24	200–350
Fast text	M1	Qwen 2.5 7B	Q4_K_M	15–18	24	130–220
Fast text	M4	Llama 3.1 8B	Q4_K_M	25–33	10	200–330
Reasoning	M4	Qwen 2.5 14B	Q4_K_M	12–18	7	85–125
Reasoning	M3	Qwen 2.5 14B	Q4_K_M	9–14	7	65–100
Vision (text gen)	M3/M4	MiniCPM-V 2.6	Q4_K_M	10–18	6	60–110
Embedding	M1/M3/M4	BGE-M3	FP16	N/A	Batch 256	~1,500 items/s
Speech	M1	Whisper L v3 Turbo	Compressed	RT	10+ streams	20+ concurrent

Notes: - Aggregate tok/s = single_request_tok/s × effective_concurrent (slightly sublinear due to batching overhead) - Vision TTFT adds 500ms–2s for image encoding before first text token

16.2 Cluster-Level Throughput Summary

Fast Pool (20 nodes):

Assuming mix: 10× M4, 6× M3, 4× M1 overflow — Qwen 2.5 7B Q4_K_M

```

10 M4 nodes × 350 agg tok/s = 3,500 tok/s
6 M3 nodes × 275 agg tok/s = 1,650 tok/s
4 M1 nodes × 175 agg tok/s = 700 tok/s
-----
Fast pool total                = 5,850 tok/s

```

At 1,000 concurrent users each requesting 30-token responses: - Sustained: 30,000 tokens needed → ~5.1 seconds total service time at 5,850 tok/s - Per-user average wait (queuing theory, M/M/1 approximation): <1s at 50% pool utilization

Reasoning Pool (8 nodes):

Assuming 4× M4, 4× M3 — Qwen 2.5 14B Q4_K_M

```

4 M4 nodes × 100 agg tok/s = 400 tok/s
4 M3 nodes × 80 agg tok/s = 320 tok/s
-----
Reasoning pool total           = 720 tok/s

```

Vision Pool (6 nodes):

Assuming 2× M4, 4× M3 — MiniCPM-V 2.6

```

2 M4 nodes × 90 agg tok/s = 180 tok/s
4 M3 nodes × 75 agg tok/s = 300 tok/s
-----
Vision pool total           = 480 tok/s (text component)
Image encoding capacity: ~12 images/s across pool

```

Embedding Pool (4 nodes):

```

4 nodes × 1,500 items/s = 6,000 embeddings/s
At batch size 256, latency < 200ms per batch

```

Speech Pool (2 nodes):

```

2 nodes × 10 concurrent streams = 20 simultaneous transcription streams
Average audio duration 30s → processed in 0.46s per stream
Effective throughput: ~40 audio files per second processed

```

16.3 User Experience Latency Targets

Workload	TTFT target (p50)	TTFT target (p95)	Streaming rate
Fast text (chat)	<1s	<3s	25–35 tok/s
Coding (fast pool)	<2s	<5s	25–35 tok/s
Reasoning	<3s	<8s	12–18 tok/s
Vision	<5s (includes encoding)	<12s	10–18 tok/s
Embedding (single)	<50ms	<200ms	N/A
Embedding (batch 256)	<200ms	<500ms	N/A
Speech (30s audio)	—	<1s total	N/A

These targets are achievable at ≤50% pool utilization. At 80%+ utilization, p95 latency will increase by 2–3×. The cluster is sized at 2× headroom for the stated 1,000-user load to maintain these targets.

16.4 Capacity Expansion Path

When utilization consistently exceeds 70% of any pool, the expansion options in order of preference:

1. **Rebalance node allocation:** Drain M1 nodes from embedding pool (which has headroom) and move to fast pool. Embedding pool can sustain higher load with fewer nodes.
 2. **Add nodes to cluster:** Mac Mini nodes can be added without cluster reconfiguration. Register in pool, restart with model, traffic begins.
 3. **Migrate to 24 GB nodes:** Replace 16 GB nodes with Mac Mini M4 24 GB variant. This doubles KV cache budget per node, enabling 30B models or 2× concurrent capacity at 14B.
 4. **Activate distributed pool:** Temporarily expand reasoning capacity using distributed EXO pool (3–4 nodes) for 30B-class models during peak demand.
-

Appendix A: Technology Stack Summary

Layer	Technology	Version (minimum)	Notes
Inference runtime (text)	vLLM-MLX	0.2.6+	Continuous batching, paged KV cache
Inference runtime (speech)	WhisperKit	Latest stable	ANE acceleration
Embedding serving	mlx-lm or sentence-transformers	Latest	BGE-M3 only
Distributed inference (experimental)	EXO	1.0+	Pipeline parallel, Mac cluster
API gateway	LiteLLM Proxy	1.x	Self-hosted, governance enforcement
Load balancer	HAProxy	2.8+ LTS	Layer-4, fast failover
HA VIP	keepalived	2.2+	VRRP-based VIP migration
Rate limit / state	Redis	7.0+	Sorted-set sliding window
Model registry	PostgreSQL	16	Model approval workflow
Model artifact store	MinIO	AGPL or commercial	S3-compatible, replicated
Metrics	Prometheus	2.45+	15s scrape, 30-day retention
Dashboards	Grafana	10+	4 dashboards as specified
Alerting	Alertmanager	0.26+	Slack + email routing
System metrics	node_exporter	1.7+	Per-node memory/CPU/disk
Container runtime	Docker or Podman	Docker 24+	Gateway, Redis, Prometheus, PG

Layer	Technology	Version (minimum)	Notes
Service discovery	mDNS (Bonjour) or Consul	Native / 1.17+	Node pool registration
Operating system	macOS	Ventura (13) or later	Required for MLX
Inference framework	Apple MLX	0.18+	Metal acceleration

Appendix B: Operational Runbook Reference

B.1 Adding a New Inference Node

```
# 1. Install macOS + developer tools
xcode-select --install

# 2. Install Python 3.11+
brew install python@3.11

# 3. Install vLLM-MLX
pip install vllm-mlx

# 4. Configure node identity
cat > /etc/hydra/node.conf <<EOF
NODE_ID=fast-node-21
POOL=fast
MODEL_ID=qwen-2.5-7b-instruct
MODEL_VERSION=v1.0
MODEL_QUANT=q4_k_m
MINIO_ENDPOINT=http://10.0.20.50:9000
MINIO_ACCESS_KEY=<read-only-key>
MINIO_SECRET_KEY=<read-only-secret>
REGISTRY_DB_URL=postgresql://readonly:pass@10.0.20.62/hydra
EOF

# 5. Run model pull + SHA256 verify script
/opt/hydra/bin/pull-model.sh

# 6. Start inference service
launchctl load /Library/LaunchDaemons/ai.hydra.inference.plist
# OR: systemctl start hydra-inference

# 7. Verify health
curl http://localhost:8000/health

# 8. Node auto-registers via mDNS; traffic begins within 30s
```

B.2 Rolling Model Update

```
# 1. Upload new model version to MinIO
mc cp new-model.gguf hydra/llm-models/qwen-2.5-7b-instruct/v1.1/

# 2. Register new version in registry (sets approved=false)
psql $REGISTRY_DB_URL -c "INSERT INTO model_versions ..."

# 3. Submit approval request
hydra-admin model approve --version-id <uuid>

# 4. Second operator approves
hydra-admin model approve-confirm --version-id <uuid> --approver ops-lead

# 5. For each node in pool (rolling, 2 at a time):
hydra-admin node drain --node-id fast-node-01 --timeout 30
# (wait for drain)
hydra-admin node set-model --node-id fast-node-01 --version-id <uuid>
launchctl unload /Library/LaunchDaemons/ai.hydra.inference.plist
# pull-model.sh re-runs on next start (version mismatch detected)
launchctl load /Library/LaunchDaemons/ai.hydra.inference.plist
hydra-admin node activate --node-id fast-node-01
# (wait 3 health checks = 30s)
# Repeat for next node
```

Appendix C: Capacity Reference Card

Cluster: 40 nodes (+ infrastructure)
 RAM per node: 16 GB
 Available per node (after OS + runtime): ~13.4 GB

Per-node weight ceilings:

7B Q4_K_M → 4.8 GB weights, 8.6 GB KV budget
 8B Q4_K_M → 5.2 GB weights, 8.2 GB KV budget
 14B Q4_K_M → 9.0 GB weights, 4.4 GB KV budget

KV cache per token (reference models):

Llama 3.1 8B → 0.125 MB/token
 Qwen 2.5 7B → 0.055 MB/token (GQA, 4 KV heads)
 Qwen 2.5 14B → 0.188 MB/token

Operational concurrent requests (4K context):

Llama 3.1 8B per node → 10
 Qwen 2.5 7B per node → 24
 Qwen 2.5 14B per node → 7

Cluster capacity (fast pool, 20 nodes, Qwen 2.5 7B):

200-480 simultaneous requests
 5,000-8,000 tokens/second sustained

Peak demand model:

1,000 users × 10% concurrency = 100 simultaneous requests
 Headroom: 2-4.8×

Distributed inference (30B, 3 nodes):

~8 tok/s single-request, ~24 tok/s aggregate
 Off-peak experimental only

Single-node impossible: any model requiring >9.6 GB weights at serving quantization